上海大学
SHANGHAI UNIVERSITY

本科毕业论文（设计）

UNDERGRADUATE THESIS (PROJECT)

题　　目：糖尿病患者日常健康管理与随访系统

学　　院：通信与信息工程学院

专　　业：电子信息工程

学　　号：21123686

学生姓名：王彦恺

指导教师：张麒

起讫日期：2025.02-2025.06

姓　　名：王彦恺　　学号：21123686

论文题目：糖尿病患者日常健康管理与随访系统

原 创 性 声 明

本人声明：所呈交的论文是本人在指导教师指导下进行的研究工作。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已发表或撰写过的研究成果。参与同一工作的其他同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

签 名：王彦恺　日 期：2025.5.20

本论文使用授权说明

本人完全了解上海大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（保密的论文在解密后应遵守此规定）

签 名：王彦恺　指导教师签名：　日期：2025.5.20

摘　要

糖尿病作为一种全球性慢性疾病，其复杂的日常自我管理以及医疗资源的相对不足对患者健康和社会带来了持续挑战。本论文聚焦于提升糖尿病管理的效率与个性化水平，设计并实现了一个智能化的糖尿病患者日常健康管理与医生随访系统。

系统前端采用用户基础广泛、轻量便捷的微信小程序，实现了用户认证、多维度健康数据（血糖、胰岛素、饮食等）的记录、可视化展示、医患实时聊天以及随访关系管理等功能，注重优化用户体验和数据录入效率。系统后端基于高性能 Python 网页框架 FastAPI 构建，具备处理高并发请求及灵活扩展的能力；数据存储创新性地采用了 MySQL 与 MongoDB 的混合模式，分别存储结构化用户档案、医患关系数据以及高频生成的半结构化日常记录和聊天消息，兼顾数据一致性与处理效率。本系统还集成了一个基于递归自注意力深度网络的个性化短期血糖预测模块。该模型借鉴了变换器（Transformer）在处理序列数据上的优势，并针对血糖数据特性进行了优化，能够从历史多模态健康数据中学习复杂的生理规律。为提升模型在个体数据有限时的预测精度，本论文采用了先在群体数据上预训练、再在目标患者数据上微调的迁移学习策略。在 OhioT1DM 数据集上进行的实验结果表明，所使用的个性化预测方法在未来 30 分钟和 60 分钟血糖预测任务上展现出优越性能，通过定量的消融实验验证了非血糖特征（胰岛素、碳水化合物摄入）及迁移学习对模型性能的显著贡献。

本论文成功构建了一个集便捷数据管理、高效医患互动与智能预测预警于一体的系统原型，为基于信息技术和人工智能改进糖尿病管理模式提供了有价值的实践探索。

关键词：糖尿病管理；血糖预测；微信小程序；人工智能；随访系统

ABSTRACT

Diabetes, as a global chronic disease, poses continuous challenges to patients' health and society due to its complex daily self-management and relatively insufficient medical resources. Focusing on improving the efficiency and personalization of diabetes management, this thesis designed and implemented a smart daily health management and doctor follow-up system for diabetes patients.

The system utilizes the WeChat mini-program as a widely adopted, lightweight, and convenient user interface, realizing functions such as user authentication, recording of multi-dimensional health data (blood glucose, insulin, diet, etc.), data visualization, real-time doctor-patient chat, and follow-up relationship management, with an emphasis on optimizing user experience and data entry efficiency. The backend of the system is built upon the high-performance Python web framework FastAPI, possessing the capability to handle high-concurrency requests and offering flexible scalability. Data storage innovatively employs a hybrid approach combining MySQL and MongoDB, storing structured user profiles and doctor-patient relationship data in MySQL, while managing high-frequency, semi-structured daily records and chat messages in MongoDB, balancing data consistency with processing efficiency. The core innovation of this system lies in the deep integration of a personalized short-term blood glucose prediction module based on a Recurrent Self-Attention Network. This model leverages the advantages of the Transformer model in processing sequence data and is optimized for the characteristics of blood glucose data, capable of learning complex physiological patterns from historical multi-modal health data. To enhance the prediction accuracy when individual data is limited, this study adopted a transfer learning strategy involving pre-training on collective data followed by fine-tuning on the target patient's data. Experimental results on the OhioT1DM dataset demonstrate that the personalized prediction method exhibits superior performance in predicting blood glucose levels 30 and 60 minutes into the future. Quantitative ablation studies verified the significant contributions of non-glucose features (insulin, carbohydrate intake) and the application of transfer learning to model performance.

This thesis successfully constructed a system prototype integrating convenient data

management, efficient doctor-patient interaction, and intelligent prediction and early warning, providing a valuable practical exploration for improving diabetes management models based on information technology and artificial intelligence.

Keywords: Diabetes management; Blood glucose prediction; WeChat mini-program; Artificial intelligence; Follow-up system

目　录

第一章 引言 1
1.1 研究背景 1
1.2 研究现状 2
1.2.1 前端研究现状：糖尿病管理应用与微信小程序 2
1.2.2 后端研究现状：主流后端框架与 FastAPI 技术选择 3
1.2.3 算法研究现状：血糖预测方法演进 5
1.3 研究目的与意义 6
1.4 论文创新点 8
1.5 论文组织结构 9
第二章 微信小程序前端设计与实现 11
2.1 微信小程序技术概述 11
2.2 系统前端架构设计 11
2.2.1 小程序整体模块划分与结构组织 11
2.2.2 页面导航设计 13
2.2.3 数据流动设计 13
2.2.4 前端与后端 API 交互机制 13
2.3 主要功能模块实现 14
2.3.1 用户认证与信息管理模块 14
2.3.2 实时聊天功能与 WebSocket 客户端 17
2.3.3 医患随访模块 19
2.3.4 数据展示与录入模块 22
2.3.5 数据可视化展示模块 25
2.3.6 血糖预测结果展示模块 27
2.4 代码质量分析 30
2.5 本章小结 30
第三章 后端与数据库设计与实现 32
3.1 后端技术选型与概述 32

3.2 后端系统架构设计 33
3.3 数据库设计与实现 35
3.3.1 MySQL 数据存储设计 35
3.3.2 MongoDB 数据存储设计 36
3.3.3 混合数据库架构与协同策略 37
3.4 后端核心模块实现 38
3.4.1 用户管理与认证接口 38
3.4.2 健康数据记录与查询接口 39
3.4.3 血糖预测接口 40
3.4.4 实时通信接口 41
3.5 本章小结 42
第四章 血糖预测算法设计与实现 44
4.1 数据集介绍 44
4.1.1 糖尿病数据集综述 44
4.1.2 OhioT1DM 数据集结构与选用原因 45
4.2 时间序列预测模型基础与 Transformer 模型概述 45
4.3 数据预处理与特征构建 46
4.4 递归自注意力网络模型架构 47
4.4.1 自注意力层 48
4.4.2 模型整体结构 49
4.4.3 迁移学习策略 49
4.5 实验设计与模型训练 50
4.5.1 数据集与划分 50
4.5.2 实验设置 50
4.5.3 模型训练 50
4.5.4 评估指标 51
4.6 实验结果与分析 51
4.6.1 主要实验结果 52
4.6.2 消融研究 1：非血糖特征的影响 53
4.6.3 消融研究 2：迁移学习的影响 54
4.6.4 局限性讨论 55

4.7 本章小结 55

总结与展望 56

5.1 总结 56

5.2 展望 56

参考文献 58

致　谢 60

第一章 引言

1.1 研究背景

糖尿病已成为 21 世纪全球范围内的重要公共卫生挑战。据国际糖尿病联合会最新发布的第 11 版《糖尿病地图》数据显示，全球约每九个成年人中就有一人患有糖尿病，患者总数高达 5.89 亿之巨[1]。更令人担忧的是，仍有许多患者尚未被诊断，从而错失早期干预的宝贵机会，导致严重并发症风险升高。在中国，糖尿病流行情况同样严峻。近年来我国糖尿病患病率迅速攀升，根据中国疾病预防控制中心全国调查结果，2018 年中国成人糖尿病患病率已达 12.4%[2]。据估算，目前我国大约有 1.41 亿糖尿病患者，占全球患者总数的四分之一[3]。中国已成为全球糖尿病患者数量最多的国家，糖尿病所致的健康损害和医疗负担日益凸显。

糖尿病的危害主要源于其长期慢性并发症，但这些并发症在相当程度上是可以通过有效的日常管理予以预防或延缓的。良好的血糖控制可显著降低视网膜病变、肾病、心血管疾病等并发症的发生风险。然而，糖尿病的日常管理极具挑战性，需患者在饮食控制、规律运动、血糖监测和药物治疗等方面持续投入精力并遵循医嘱，意即患者需具备很好的依从性，对自我管理具有高要求。事实上，由于专科医生资源短缺、医疗资源地区分布不均，许多患者难以及时获得指导和随访支持；同时，一些患者缺乏足够的糖尿病知识或管理动力，导致自我管理不到位。此外，在传统模式下，患者日常产生的血糖、饮食、运动等数据往往分散在血糖仪、日记本或不同应用中，数据离散难以统一管理，医生也难以及时获取患者全面的状态信息，医患沟通存在明显不畅。这些因素综合导致相当一部分糖尿病患者血糖控制不理想，糖尿病相关的可避免发病率和死亡率仍然居高不下[4]。

值得庆幸的是，移动互联网技术的迅猛发展为改善慢病管理模式带来了契机。智能手机的普及和无线网络的覆盖使得患者能够随时随地记录健康数据、获取医疗信息和接受远程指导成为可能。尤其在中国，基于移动应用的健康管理服务近年发展迅速，“互联网+医疗健康”的模式已广泛应用于慢性病随访管理中。与此同时，人工智能（Artificial Intelligence, AI）技术在医疗领域的突破为慢病管理注入了新的活力。AI 擅长从海量健康数据中提取有意义的模式和知识，能为个体

化诊疗和预测预警提供支持。对于糖尿病这种需持续监测和决策的疾病，AI 有潜力在风险评估、血糖预测、决策辅助等方面发挥重要作用[4]。

综上所述，在全球和中国糖尿病流行严峻的背景下，如何利用移动互联网手段加强患者日常健康管理，并结合 AI 技术提高管理的效率和智能化水平，已成为糖尿病防治亟待解决的重要课题。

1.2 研究现状

1.2.1 前端研究现状：糖尿病管理应用与微信小程序

随着移动健康的兴起，大量面向糖尿病管理的移动应用（Application, App）和平台在国内外相继涌现。这些应用通常提供多种功能，帮助患者进行日常的自我管理。据文献调研，大多数糖尿病管理应用具备全面的记录和分析功能，允许用户记录血糖、饮食、运动及服药等各类数据，并对数据进行统计分析，帮助患者直观地了解自身病情变化。不少应用还能通过蓝牙等方式与血糖监测设备连接，实现测量数据的自动上传，减少人工输入的麻烦。在数据累积的基础上，应用程序往往提供基于记录数据的建议反馈或治疗支持功能，例如根据血糖水平提出饮食调整建议，或在检测到异常时提示用户复测或就医。同时，为了加强患者与医护之间的联系，不少应用增加了数据分享和通讯模块，患者可以将自己的血糖日志一键发送给医生查看，医生也可通过应用与患者进行消息交流。总体来说，这些移动应用的出现大大便利了糖尿病的自我管理，有研究表明使用糖尿病管理 App 能够有效降低患者的糖化血红蛋白（Hemoglobin A1c, HbA1c）水平、改善用药依从性并提升患者的自我管理信心[5]。

然而，现有糖尿病管理 App 在实际应用中也存在一些不足和挑战。首先，长期使用率的问题突出。有调查发现，许多患者对应用的新鲜感会随着时间减退，使用频率呈下降趋势[5]。一些用户在初始阶段热衷记录，但数周或数月后可能逐渐放弃，这使应用难以发挥持续作用。造成这一现象的原因可能在于应用设计未能充分满足用户需求或缺乏有效的机制来保持用户粘性。另外，一些应用功能相对孤立，未能与临床照护有效结合。例如，应用提供的数据医生未必能够及时获取和解读，或应用的内容未严格依据权威的糖尿病管理指南，这在一定程度上限制了其临床价值。此外，不同应用之间的数据标准不统一，也导致患者更换或同

时使用多个应用时数据无法汇总，形成新的“信息孤岛”。因此，如何提升糖尿病管理应用的用户黏性、临床融入度和数据互通性，是当前研究和业界实践关注的重点。

在我国的移动医疗领域，微信小程序的兴起为慢病管理提供了新的解决方案。微信小程序是基于微信生态的轻量级应用，自 2017 年推出以来发展迅猛。凭借无需下载安装、即用即开的便利性和覆盖超过 12 亿月活跃用户的庞大平台，微信小程序在国内拥有其他独立 App 难以比拟的推广优势。对于中老年糖尿病患者而言，微信已经是日常使用最频繁的手机应用之一，在微信内嵌入健康管理功能可以大幅降低使用门槛。这种背景下，不少糖尿病管理服务开始尝试以微信小程序作为前端载体。微信小程序在医疗健康领域的应用价值也日益凸显——有研究指出，基于微信小程序的移动健康管理平台已在国内广泛用于健康促进和自我管理，并展现出良好的用户接受度[6]。

尤其在疫情期间及后疫情时代，微信小程序在医疗服务中的渗透率进一步提高，大量医院和机构通过小程序提供预约挂号、报告查询、线上咨询等功能，培养了用户使用习惯。在糖尿病管理方面，目前国内已有基于微信小程序的管理工具投入实践。一项针对妊娠期糖尿病患者的研究表明，使用微信小程序构建血糖管理模型可以显著降低产妇餐后血糖水平，改善患者自我管理能力和满意度[7]。这种基于微信的干预模式验证了微信小程序在慢病管理中的可行性和有效性[7]。当然，微信小程序也有其局限性，如功能上需受限于微信平台的权限，复杂计算需由后台服务器支持，但总体而言，其在国内医疗场景下的独特价值已得到认可。

综上，利用微信小程序作为糖尿病日常管理的前端具有巨大的潜力，但如何充分发挥其优势、弥补其不足，还需要进一步的研究和实践探索。

1.2.2 后端研究现状：主流后端框架与 FastAPI 技术选择

在构建糖尿病管理与随访系统的过程中，后端服务框架的选择至关重要。一个优秀的后端框架需要同时满足高并发数据处理、可靠的安全性、良好的可扩展性以及方便的开发维护等要求。纵观目前主流的后端开发技术，已经形成了多种成熟的框架方案。

在 Python 生态中，Django 和 Flask 是历史较为悠久且应用广泛的两个网页（Web）框架：

（1）Django 提供了“开箱即用”的完整解决方案，内置 ORM、管理后台等功能，适合大型项目的快速开发；

（2）Flask 则以轻量灵活著称，开发者可以根据需要自由选择组件，更适合小型服务或微服务架构。

这两种框架在医疗信息系统开发中已有大量实践案例。但传统框架往往基于阻塞式的工作模型，难以充分利用异步 IO 带来的性能优势。随着实时通信和高并发需求的增加，近年来新兴的异步框架开始受到关注。在这一背景下，Python 领域出现了基于异步服务器网关接口（Asynchronous Server Gateway Interface, ASGI）的新一代高性能 Web 框架，其中尤以 FastAPI 为代表。

FastAPI 由 Sebastián Ramírez 于 2018 年推出，很快因其卓越性能和易用性而广受欢迎[8]。FastAPI 建立在高性能异步框架 Starlette 和数据模型验证库 Pydantic 之上，充分利用了 Python 的类型提示功能来简化开发过程。它最大的特点之一是对异步编程的原生支持，以及自动生成交互式 API 文档的能力：开发者只需用标准 Python 类型注解定义接口函数，FastAPI 即可自动生成遵循 OpenAPI 规范的接口文档和测试页面。这种特性极大地方便了前后端联调和第三方系统的集成，减少了开发人员编写文档的工作量。

更重要的是，得益于底层异步架构和高效的请求处理，FastAPI 的性能表现出色，可媲美 Node.js 等流行后端技术，在处理高并发 API 请求时具有明显优势。据官方介绍和社区测试，使用 FastAPI 构建的服务在延迟和吞吐量方面远优于传统的 Flask 等框架，在多数场景下接近于 Go 等编译型语言框架的性能水平，这使得 FastAPI 非常适合构建需要实时响应的 RESTful API 服务。

另外，FastAPI 高度模块化和可扩展的设计也便于与 AI 模型部署结合：例如，可以方便地将机器学习模型加载到 FastAPI 应用中，通过应用程序接口（Application Program Interface, API）提供预测服务，而异步特性允许在不阻塞主线程的情况下完成模型推理等耗时操作。综合考虑以上因素，FastAPI 已成为近年来社区最受关注的 Python 后端框架之一，在 GitHub 上累积了超过 6 万星标，展现出蓬勃的生命力。

本课题在比较多种方案后，选择 FastAPI 作为后端开发框架，正是看中了其高性能、高并发支持以及在构建数据接口和整合 AI 组件方面的优势。这一选择

有助于我们实现一个响应迅速、架构灵活的后端系统，为前端小程序和算法模块提供有力支撑。

1.2.3 算法研究现状：血糖预测方法演进

血糖水平的预测是糖尿病管理中的一项关键技术，被视为一个典型的时间序列预测问题。通过对血糖随时间变化的趋势进行建模和预测，医生和患者可以提早发现异常并采取干预措施，从而降低急性并发症（如低血糖昏迷、高血糖酮症）的风险。血糖预测的研究经历了从传统统计方法到现代人工智能方法的演进。

早期的研究多采用基于生理模型或统计学的手段。例如，一些方法假设血糖变化呈现一定的线性或周期模式，应用常规的时间序列分析模型进行预测，其中经典代表是自回归积分滑动平均模型（Auto Regressive Integrated Moving Average, ARIMA）等传统统计模型。这类模型在一定程度上能捕捉短期的血糖波动，但由于血糖调控过程高度非线性、受多因素影响，单纯线性模型的预测精度受到限制。当只利用过去的血糖值进行预测时，传统 ARIMA 模型往往难以充分利用隐含的动态特征[9]。

相反，随着机器学习技术的发展，数据驱动的预测方法开始显示出优势。其中，基于深度学习的循环神经网络（Recurrent Neural Network, RNN）及其变种长短期记忆网络（Long Short-Term Memory, LSTM）因擅长处理时间序列依赖，被广泛应用于血糖预测任务。LSTM 能够在序列数据中捕捉长程相关性，记忆先前重要的信息以影响后续预测。这种能力非常适合刻画血糖随生理节律、进食、胰岛素等因素作用下的延迟效应。实验证明，在仅使用历史血糖值进行预测的场景下，LSTM 模型的效果显著优于传统 ARIMA 模型[9]。例如，Marling 等人的研究比较了 LSTM 和 ARIMA 用于 30 分钟血糖预测的性能，结果显示 LSTM 的预测误差远小于 ARIMA，表明 LSTM 能够更有效地利用过去数小时的血糖信息来提升预测准确度[9]。此外，通过将进餐记录、胰岛素给药等信息与血糖数据一同输入模型，LSTM 还能进一步提高中长期预测的精度。

近几年来，以变换器（Transformer）为代表的新型深度学习模型在时间序列领域展现出强大的建模能力，也被逐步引入到血糖预测问题中。Transformer 最初应用于自然语言处理任务，依靠自注意力（Self-Attention）机制来高效捕捉序列中元素之间的长程依赖关系。相比 RNN 逐步迭代的方式，Transformer 可以在更

深的层次上并行处理序列并关注全局模式，这非常契合血糖序列中可能存在的日周期规律或跨餐时段模式等长时间依赖特征。一些研究者将 Transformer 架构改进用于时间序列预测，发现其在处理复杂、多变量的慢病数据时相比传统机器学习技术具有潜在优势[10]。例如，有学者提出将 Transformer 与 LSTM 相结合，构建混合模型用于血糖预测。Transformer 部分捕获长周期的趋势变化，LSTM 部分捕捉短期局部模式，两者优势互补。基于真实患者的连续血糖监测数据进行验证，这种混合模型在长达 120 分钟的预测范围内取得了较高的精度和稳定性，结果具有良好的临床适用性**错误!未找到引用源。**。具体而言，该模型在 30 分钟、60 分钟等不同预测时长上的均方根误差均显著优于以往的方法，Clarke 误差网格分析显示大部分预测落在临床无风险区域**错误!未找到引用源。**。这表明 Transformer 引入后，模型能够更有效地抓住血糖波动的全局模式，提高了长期预测的准确性。除了 Transformer 之外，学者们也尝试了诸如时序卷积网络（Temporal Convolutional Network, TCN）、图神经网络（Graph Neural Network, GNN）等新技术来改进血糖预测性能，但总体来看，基于 LSTM 和 Transformer 的深度学习方法目前代表了该领域的前沿进展。在传统方法与 AI 方法的对比中可以看到，前者如线性回归、ARIMA 等在建模非线性、生理复杂性方面力有不逮，而后者通过学习历史数据中的隐含模式展现出更佳的效果。当然，AI 方法也面临如训练数据需求大、模型可解释性不足等挑战，但随着数据积累和算法改进，这些问题正逐步得到改善。

1.3 研究目的与意义

针对前述研究现状所揭示的当前糖尿病管理在前端用户体验优化、后端系统对诸如人工智能等新兴技术的支撑能力，以及血糖预测算法精准度与个性化方面的挑战与发展机遇，本论文旨在设计并实现一个面向糖尿病患者日常健康管理与医生随访支持的综合系统。本系统以患者为中心，利用移动端工具提供便捷管理方式，并结合后端数据平台与智能算法，辅助临床医生进行远程监测和有效随访，期望以此构建高效的闭环式糖尿病管理新模式。具体而言，本系统的开发旨在达成以下四大核心目的：

（1）为应对现有糖尿病管理应用在用户黏性及操作便捷性方面的不足，并

充分发挥微信小程序在国内的普及优势，本系统首要目的在于为糖尿病患者提供一个便捷高效的平台，使其可随时记录血糖、饮食、运动、用药等关键日常健康数据，并能获得即时、个性化的反馈与指导；

（2）其次，为改善医护人员对院外患者状态掌握不全面的现状，本系统致力于为临床医生提供一套有效的患者管理工具，使其能够实时查阅患者的自我管理数据，从而进行及时干预和有效随访，进而提高医疗服务的连续性与覆盖广度；

（3）再者，针对传统血糖预测方法在准确性上的局限以及新兴人工智能算法（尤其是基于注意力机制的模型）展现出的潜力，本研究的核心目标之一是将先进的数据分析与人工智能技术深度融入慢病管理流程。通过对患者历史健康数据的深入挖掘与模式学习，系统旨在实现对血糖变化趋势的精准预测及潜在风险的智能预警，从而显著提升糖尿病管理的智能化与个体化水平；

（4）最后，为打破传统模式下医患沟通不畅、信息传递滞后的壁垒，本系统着力构建一个便捷高效的医患沟通渠道。通过在线即时交流、结构化数据共享及消息提醒等功能，强化对患者院外管理行为的支持与健康教育，以此增强其自我管理的效能感与信心。

本论文工作的开展具有重要的理论意义与实际应用价值。

首先，在患者层面，本系统通过提供更贴合用户习惯（特别是微信生态）且功能全面的管理工具，有望突破现有应用在长期使用率和用户依从性方面的瓶颈，从而通过激励患者持续记录和遵循医嘱，显著提高其自我管理的有效性，并最终改善血糖控制、降低 HbA1c 水平[5]及远期预后。

其次，在医护层面，本系统所提供的系统化患者数据平台，有助于克服之前提及的数据分散与“信息孤岛”问题，使医生能更全面、及时地掌握患者院外的血糖波动及生活方式依从情况。这不仅使得远程干预和个性化指导成为可能，显著提高了随访管理的效率，亦能在一定程度上缓解当前优质医疗资源相对紧缺及分布不均的矛盾。

再次，本系统通过结构化地收集和整合丰富的日常管理数据，为人工智能算法在糖尿病管理中的深度应用创造了坚实基础。利用这些真实世界数据训练的 AI 模型，能够为医患双方提供更精准的个体化决策支持，例如预测高、低血糖风险，评估不同干预措施的潜在效果等。这种数据驱动的智能管理模式，相较于

传统经验性判断，有望大幅提升慢病管理的科学性、前瞻性与精准度。

最后，针对传统医患沟通模式存在的时空限制与信息不对称问题，本系统构建的便捷互动渠道，通过在线咨询、数据共享和针对性健康教育推送等功能，极大地促进了医患间的有效沟通。这种持续的互动不仅能增强患者的疾病认知与自我管理技能，提升其就医获得感与信任感，更有助于构建和谐、高效的新型医患协作关系。

1.4 论文创新点

本论文在深入分析现有糖尿病管理系统及相关技术研究的基础上，针对其在提升用户参与度与数据管理便捷性、构建适应复杂健康数据与 AI 集成需求的高效后端、以及实现精准个性化智能预测等方面存在的局限与挑战，进行了多维度的创新性探索与实践。主要的核心创新性主要体现在以下几个方面：

第一，构建了深度融合微信生态的患者赋能与医患协同新模式。 针对传统独立 App 常面临的用户学习成本高、数据易成孤岛及长期使用率不足等问题，本系统独辟蹊径，以微信小程序为核心前端。此选择的创新性不仅在于利用其“即用即走”特性显著降低了患者的使用门槛，更关键在于将多维度健康数据（血糖、胰岛素、饮食、运动等）的便捷记录、直观可视化、实时医患沟通及个性化随访管理等功能深度整合于用户日常高度依赖的社交生态内。这种设计旨在打造一个低学习曲线、高依从性的糖尿病日常管理闭环，有效促进患者的持续主动参与，并极大提升了医患双方信息同步与协作的流畅度与效率。

第二，设计并实现了一种面向糖尿病管理特性、支持混合异构数据高效处理与 AI 无缝集成的协同后端架构。为有效应对糖尿病管理过程中结构化核心数据（如用户档案、医患绑定关系）与高频产生的半/非结构化健康数据（如日常记录、聊天日志、生理序列）并存，且需高性能处理及支持 AI 模块运算的复杂需求，本系统后端架构进行了创新性设计。采用了基于 FastAPI 的高性能异步框架，并独创性地结合了 MySQL 关系型数据库与 MongoDB 文档型数据库的混合存储策略。此架构的创新价值在于，它不仅通过异步机制保障了对高并发请求的快速响应，通过关系型数据库确保了核心业务数据的完整性与一致性，更利用文档数据库的灵活性与高效读写特性优化了对海量、多样化健康记录的处理。同时，清

晰的分层与模块化设计，以及标准化的 API 接口，为后续先进血糖预测等 AI 模块的无缝集成、高效训练与快速迭代部署提供了坚实的技术支撑与高度的可扩展性。

第三，实现了基于深度学习的个性化血糖预测模型与日常管理流程的闭环式深度集成。 针对传统血糖预测方法在精度和个性化方面的不足，以及现有管理工具多停留在数据记录与展示层面、缺乏前瞻性智能决策支持的现状，本系统在智能化应用方面取得了关键突破。核心创新在于将先进的、基于递归自注意力网络的个性化短期血糖预测模型，深度嵌入到从数据采集、模型推理到结果反馈的完整用户管理流程之中。该模型能有效从患者多模态历史数据中学习复杂的动态生理模式，并通过迁移学习策略提升个体适应性。预测结果不仅以用户友好的方式在前端呈现，更可为患者的即时行为调整（如饮食、运动决策）和医生的远程干预提供前瞻性的数据洞察。这种从“被动记录分析”到“主动智能预测与干预指导”的模式升级，是本系统在提升糖尿病管理智能化、精准化与个性化水平方面的核心贡献。

综上所述，本论文通过在前端交互与赋能模式、后端数据处理与 AI 支撑架构、以及核心智能算法的深度集成应用等多个层面的协同创新，构建了一个旨在显著提高糖尿病管理便捷性、医患协作效率和临床决策智能化水平的综合管理与随访系统。这些创新点的有机结合，不仅分别针对性地回应了现有研究与实践中的特定痛点与挑战，更共同形成了一个具有显著前瞻性和临床应用潜力的糖尿病智慧管理解决方案，为未来信息技术与人工智能在慢性病管理领域的深入融合与应用模式创新提供了有益的探索和实践范例。

1.5 论文组织结构

为了系统地阐述糖尿病患者日常健康管理与随访系统的设计与实现过程，本文共分为五章，各章节内容安排如下：

第一章介绍课题的研究背景与意义，综述国内外研究现状，明确本文的创新点，概述论文的整体结构安排。

第二章阐述微信小程序平台的技术特点及开发框架，分析系统前端的架构设计与主要功能模块实现过程，并结合实际开发过程中的功能测试与性能分析，验

证前端系统的设计合理性与运行效果。

第三章介绍后端开发所采用的技术体系，重点讲解基于 FastAPI 框架构建的后端服务架构与 RESTful API 设计原则，详细分析 MongoDB 与 MySQL 混合数据库方案的设计与实现过程。

第四章梳理时间序列预测的基本理论，解析 Transformer 模型的结构与适用性，详细叙述基于 PyTorch 框架完成的血糖预测模型构建、训练与部署过程，结合模型评估指标对预测性能进行测试与分析，并探讨 AI 模块在实际应用中的效果与优化空间。

总结与展望部分总结本系统在前端设计、后端开发、数据库管理与 AI 预测模块等方面的主要研究内容与实现成果，分析系统整体功能实现效果与技术选型的优劣，回顾项目开发过程中遇到的挑战与应对策略，探讨系统当前存在的不足，并展望未来在功能完善、模型优化及实际应用推广方面的进一步工作方向。

通过上述章节安排，本论文力图从系统开发全流程的角度，清晰呈现糖尿病患者日常健康管理与随访系统的设计思路、实现细节、技术应用与效果评估，力求为慢性病智能管理系统的研究与实践提供有益的参考和探索。

第二章 微信小程序前端设计与实现

作为整个系统与用户进行直接互动的关键组成部分，前端承担着健康数据的呈现、用户操作的捕获以及与后端服务高效通信的职责。本章将首先概述微信小程序的技术特性及其在健康管理应用中的优势，随后深入探讨前端系统的整体架构设计、模块划分、页面导航与数据流动策略，并对用户认证、数据记录与展示、医患随访、实时聊天以及血糖预测结果展示等核心功能模块的具体实现进行详细阐述。

2.1 微信小程序技术概述

微信小程序作为一种轻量级的应用形态，以其免安装、即开即用的特性，为用户提供了极大的便利，特别适用于日常健康管理这类高频、轻量级的使用场景。开发者基于微信提供的 WXML、WXSS 和 WXS 等前端技术栈进行页面构建和逻辑开发。

在渲染引擎方面，在综合考量了不同微信基础库版本的兼容性后，最终本论文还是选择采用具有更广泛兼容性的 WebView 渲染模式，以确保系统在多种用户设备上的稳定运行。界面组件方面，系统引入了腾讯 TDesign 组件库，通过其统一的设计规范和丰富的用户界面（User Interface, UI）组件，提高了开发效率，保障了用户界面的美观性与一致性。

2.2 系统前端架构设计

2.2.1 小程序整体模块划分与结构组织

系统前端设计框架如图 2.1 所示。系统前端在逻辑上根据用户角色被划分为患者端和医生端。系统根据用户登录时获取的角色信息，动态地呈现适配不同角色的功能界面和导航入口。虽然底部的 Tab 导航栏（“首页”、“消息”、“我的”）在两端保持一致，但各 Tab 页的具体内容和子功能入口会根据当前用户身份进行区分显示。三个 Tab 页的展示如图 2.2 所示。

为了优化小程序的加载性能并管理代码包体积，系统采用了微信小程序的主包与分包机制。主包承载应用的核心启动代码、公共资源及用户高频访问页面（如部分首页、消息页等）。非核心或体积较大的功能模块和不常用页面被划分到不

同的分包中，实现按需加载，有效减少了首次加载时间并突破了主包体积限制，提升了用户体验。

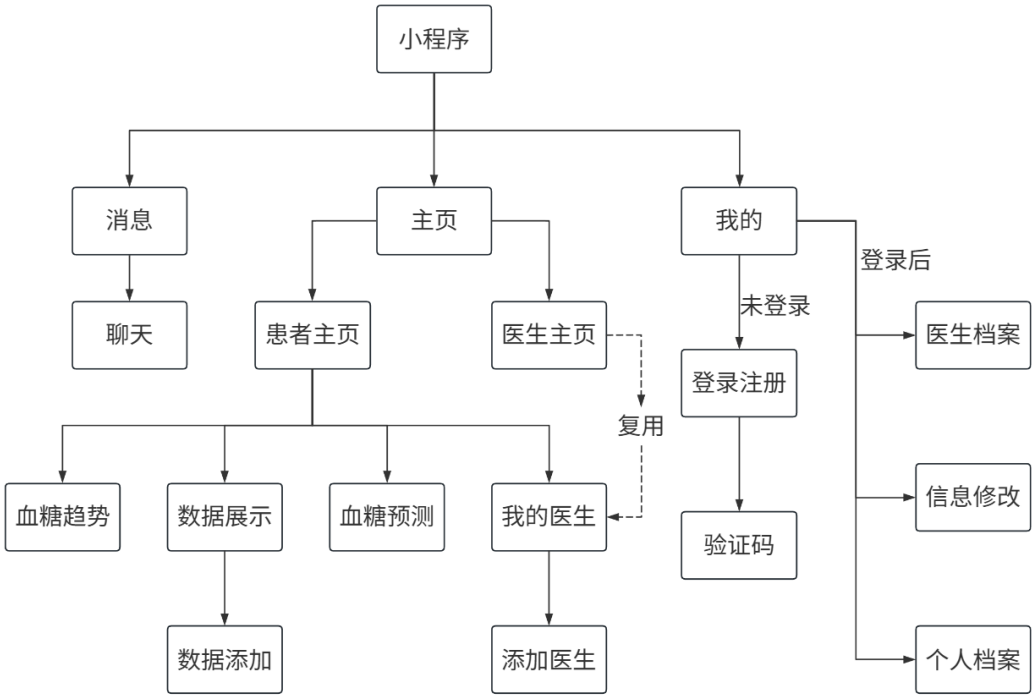
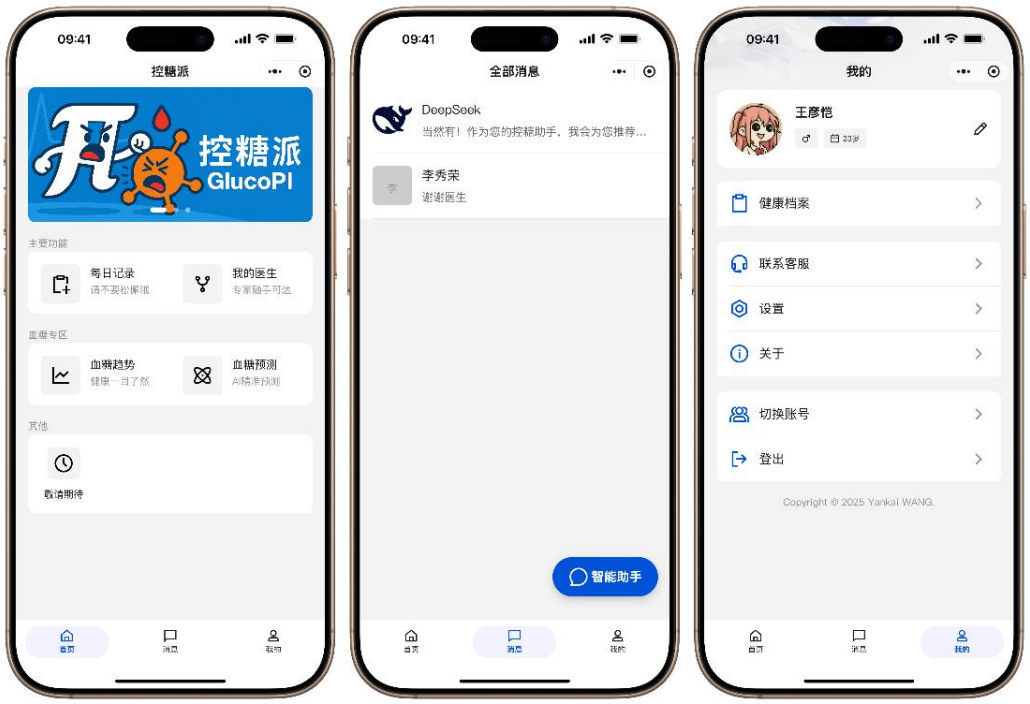


图 2.1 微信小程序架构图



（a） （b） （c）

图 2.2 小程序顶级页面。（a）首页，（b）“消息”页，（c）“我的”页。

2.2.2 页面导航设计

清晰的页面导航设计对于保障用户流畅使用至关重要。本系统采用了顶级 Tab 页与普通页面结合的页面层级结构。底部 Tab 页（首页、消息、我的）作为应用的顶级入口，通过 wx.switchTab 方法进行页面切换。由 Tab 页或其他普通页面进入的功能详情、编辑、流程页面等则作为普通页面，通常使用压入页面栈（wx.navigateTo）或替换当前页面（wx.redirectTo）等方法进行跳转。针对不同的导航场景，灵活运用这些 API，确保用户操作流程的逻辑性和顺畅性。

2.2.3 数据流动设计

前端系统内的数据流动主要包括页面间的数据传递、与后端的数据交互以及数据状态的响应更新。

页面间的数据传递采用多种策略：对于少量简单数据，利用 URL 参数随页面跳转传递；对于需要在多个页面或组件间共享且在应用生命周期内保持一致的数据，存储在全局 App().globalData 对象中（如用户信息、WebSocket 实例）；对于需要跨会话持久化或启动时即刻获取的认证信息，利用小程序本地缓存进行存储（如 Token、用户 ID、角色）。

与后端的数据交互统一通过一个封装的 request()工具方法实现。该方法基于微信小程序原生的登陆方法（wx.request），集成了 Token 自动携带、登录态失效检查与自动跳转、以及统一的错误和响应处理逻辑。数据通常在页面加载或显示时通过调用此封装方法从后端获取。登录态的维护依赖本地缓存的 Token，后端通过标准令牌机制（JSON Web Token, JWT）进行校验。

数据响应机制结合了主动拉取和实时推送。对于用户操作导致的数据更新，前端通常在操作成功后重新拉取相关数据刷新界面。对于聊天消息等需要即时送达的数据，系统利用 WebSocket 实现服务器到前端的实时推送，并通过前端的事件总线机制分发消息，确保界面的实时性。

2.2.4 前端与后端 API 交互机制

本系统前端与后端服务之间的通信基于标准的 HTTPS 协议，通过 RESTful API 进行。所有请求和响应数据均使用统一的 JSON 格式进行封装。前端通过封装的 request()方法发起 HTTP 请求，后端基于 FastAPI 框架进行处理。用户身份认证采用 JWT，前端在每个需要认证的 API 请求头中附带 Token。API 设计遵循

RESTful 规范，使用标准的 HTTP 动词和有语义的 URL。后端返回标准的 HTTP 状态码及业务自定义错误码，前端根据返回值进行统一的提示和处理。所有 API 接口路径统一加版本前缀(/api/v1/)，便于版本管理和迭代。

2.3 主要功能模块实现

本系统前端微信小程序实现了多项核心功能模块，共同支撑起糖尿病日常健康管理与医患随访的各项服务。这些模块的设计与实现直接影响用户体验，是系统价值的直接体现。本节将对各个主要功能模块的功能简介、用户交互流程等进行详细阐述。

2.3.1 用户认证与信息管理模块

(1) 功能简介

此模块作为系统整体稳定运行与用户交互的基石，其核心功能不仅涵盖了用户的注册、登录、注销等基础身份管理操作，亦包括个人信息的查阅与编辑权限。作为用户首次接触并使用系统的关键环节，该模块为所有后续需识别用户身份的系统操作构建了不可或缺的前置基础。为优化特定前端平台（如微信小程序）的初始加载性能并提升代码组织的整体效率与可维护性，系统在设计层面将相关功能依据其使用频率与耦合程度，策略性地分布于不同的界面单元及代码组织结构中。例如，作为用户个人信息核心入口的“我的”界面，及其关联的核心登录注册流程（包含手机号码输入与验证码核验等关键步骤），均被部署于系统优先加载的核心功能模块内，以确保用户能够快速访问并完成认证。相较之下，诸如账户详细信息、特定角色专属档案（例如患者档案、医生档案等）的编辑功能，因其相对较低的使用频率，则被组织到独立的、可按需加载的个人信息拓展模块之中。此种模块化的划分与加载策略，有效地降低了核心模块的初始体积，显著优化了应用的启动速度，并在模块的可访问性、加载效率与整体代码的可维护性之间取得了良好的平衡。

(2) 交互流程

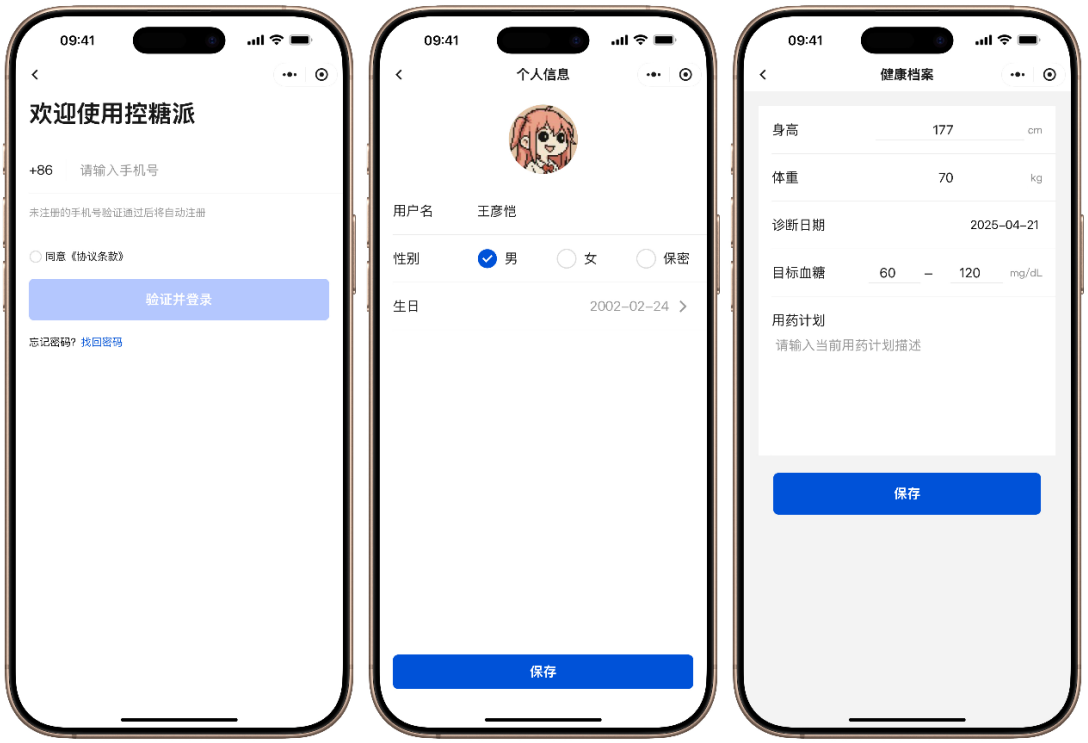
用户认证的交互流程主要依托于专门设计的登录注册界面及验证码核验界面协同完成。用户既可从“我的”界面等入口主动触发登录操作，亦可在尝试访问需要特定登录权限的系统功能时，被系统自动导航至统一的登录界面。系统的

核心认证机制建立在用户手机号码与短信验证码的双重验证基础之上：用户输入手机号码后，系统将引导其进入验证码核验界面，以完成后续的身份确认步骤。前端应用会依据用户输入的手机号码在后端的注册状态——此状态通过与后端服务的安全通讯进行查询确认——来动态决策是引导用户执行既有账户的登录流程，还是启动新用户的注册程序。为进一步提升用户体验的流畅性，系统亦支持基于微信平台开放身份标识（OpenID）的快捷登录机制。在此模式下，前端应用在获取用户的 OpenID 后，将通过与后端服务接口交互，查验该 OpenID 是否存在已关联的用户账户；若确认存在，则用户可免除手机号码验证环节，直接完成登录。用户成功登录系统后，前端应用将负责妥善维护其登录会话状态，并提供清晰的注销功能，该注销操作将清除本地存储的认证凭据并重置应用至未登录状态。

信息管理功能则主要围绕“我的”界面为核心进行组织与呈现。用户通过点选个人信息展示区域，即可导航至账户信息编辑界面，在此处可对昵称、头像等个人基础资料进行修改。若用户选择如“健康档案”等特定功能入口，系统将依据其当前已认证的角色身份（例如患者用户或医生用户），智能导航至对应的专属档案编辑界面（如患者档案界面或医生档案界面），从而允许用户便捷地查阅并修订其特定的健康档案信息内容。前端界面将依据后端服务返回的角色信息进行动态适配与渲染，以确保向不同角色的用户展示相匹配的档案字段及内容。头像上传功能的设计采用了一种独立的图像存储服务方案：前端应用通过调用平台提供的标准化文件上传接口，将用户选定的图片资源传送至该专用图像存储服务；该服务负责图像的持久化存储，并为每个图像文件生成一个通用唯一标识符（Universally Unique Identifier, UUID）作为其独特的文件名，随后将图像在云端存储的统一资源定位符（Uniform Resource Locator, URL）返回给前端应用。前端系统本身不直接存储图像的二进制实体，而是将获取到的此 URL 作为图像资源的引用标识，通过调用后端的相应 API——例如用于更新用户资料的接口——将该 URL 更新并持久化至数据库的用户信息记录中。因此，在后续的任何界面需要显示用户头像时，系统仅需利用已存储的 URL 直接加载并渲染图像资源即可，此种设计有效地实现了图像实体存储与核心业务数据的解耦。

此模块的页面展示如图 2.3 所示。分别展示了三个页面，为“登录注册”页，

“个人信息”页与“患者档案”页。



（a） （b） （c）

图 2.3 用户认证与信息管理模块。（a）“登录注册”页，（b）“个人信息”页，（c）“患者档案”页。

(3) 状态管理

前端的用户状态管理体系采用了一种多层次、职责分明的策略，旨在确保应用内各类数据的内在一致性、存储持久性及在不同上下文中的可用性与高效访问。对于在应用程序整个生命周期内均需共享的关键非持久化数据，例如当前已登录用户的完整信息对象、实时通信（如 WebSocket）连接实例等，系统将其统一存储于全局应用状态容器之中，以便于各个独立页面及功能组件间的快速、便捷访问。与之相对，高度敏感的认证相关信息，包括用以接口调用的访问令牌（access_token）、系统内部分配的用户唯一标识（user_id）以及标识用户权限的角色信息（role）等，则利用客户端本地的持久化缓存机制进行安全存储。这种设计确保了即使用户关闭并重启应用程序，系统也能够通过读取这些持久化信息自动恢复用户的登录会话状态。这些关键认证信息在应用初始化阶段即被读取以尝试恢复登录态，并在所有后续需进行身份验证的 API 请求中被自动且安全地携带。

对于特定界面内部的临时性状态数据和用户在表单中输入的值，尤其是在涉及复杂信息编辑的页面场景下，则由该界面自身的局部状态管理机制（通常表现为页面级的数据对象）进行精细化维护。通过健全的事件绑定逻辑与相应的界面更新函数（例如微信小程序中的 setData()方法或类似机制），系统能够高效实现表单输入数据与用户界面显示内容之间的双向同步与实时更新。当用户成功完成登录操作后，其核心认证信息会即时同步更新至全局应用状态容器以及本地持久化缓存中，以确保状态统一。在进行个人信息修改的交互流程中，前端应用通常首先更新当前页面的局部状态以即时响应用户操作，随后调用后端 API 执行数据的持久化保存；在接收到后端成功处理的响应后，前端会主动刷新当前页面的相关数据（一般通过重新请求后端的用户信息接口以获取最新、最准确的数据），并同步更新全局状态容器与本地持久化缓存中的对应信息，以此严谨地保障前端所展示数据与后端数据库中存储数据间的最终一致性。

2.3.2 实时聊天功能与 WebSocket 客户端

(1) 功能简介

本模块构成了系统内医患沟通及人机智能交互的核心枢纽。其主要功能涵盖了用户间的实时文本消息交换、历史消息记录的有效加载与展示，以及未读消息数量的动态管理。值得一提的是，该通信机制亦被复用以支持患者用户与集成的 AI 智能助手进行交互问答。从前端界面组织来看，系统的核心模块内承载了“消息”主页，用以聚合展示用户的会话列表及各会话的最新消息摘要。而具体的“聊天详情”界面，则作为按需加载的扩展模块，在用户选定特定会话时加载，专用于展示该会话的完整消息流，并提供消息输入功能。此种基于模块划分的加载策略，旨在优化应用的初始加载性能，并提升用户在不同功能间的切换体验。

(2) 交互流程

用户通常从“消息”主页点击某一特定会话条目，从而导航并进入对应的“聊天详情”界面。该界面的布局设计力求直观与高效，清晰地展示了对话另一方（或 AI 助手）的基本信息、按时序排列的历史消息列表，以及底部的文本输入区域。用户亦可通过系统内设定的特定入口（例如点击“智能助手”相关按钮）直接发起或进入与 AI 智能助手的专属会话界面。进入聊天详情页时，其核心交互逻辑在于建立或绑定一个稳定、持久的 WebSocket 实时通信连接。一旦连接成功建

立，该页面将注册相应的消息处理回调函数，用以监听并处理服务器推送的新消息。为避免重复注册或潜在的逻辑冲突，系统在页面层面采用了一种控制机制（例如页面作用域的标志位）来确保回调函数仅被有效注册一次。

当接收到新的传入消息时，已注册的消息处理回调函数会依据消息的来源（如发送者标识）或特定标记（例如，一个用于区分是否为 AI 智能助手生成消息的指示符）进行智能分类与差异化处理。对于 AI 助手回复的消息，系统在将其添加入消息列表前，可能会执行一系列针对性的渲染前置操作，例如调用富文本解析与渲染组件（以支持复杂的文本格式或内嵌元素显示）、计算 AI 响应耗时等，随后才更新界面列表。而普通用户间的文本消息，则一般直接按照时间顺序插入到消息列表的适当位置。此种分类处理机制确保了不同来源、不同类型的消息均能得到恰当且美观的展示，并能有效应对 AI 智能助手响应内容可能存在的特殊格式与元信息。

此模块的页面展示如图 2.4 所示。分别展示了两个页面，为“消息”页与“聊天”页。

（a）　　（b）

图 2.4 实时聊天模块。（a）“消息”页，（b）“聊天”页。

(3) 状态管理

本模块在状态管理层面，巧妙地结合了页面级局部状态与应用级全局状态两种策略，以适应不同数据的特性与生命周期需求。如前文 2.3.1 节的状态管理部分所述，全局应用状态容器负责存储用户完整信息、已建立的 WebSocket 连接实例等需在应用整体生命周期内共享的关键数据。本地持久化缓存则继续承担存储敏感认证信息的职责，用以支持应用的快速重登录或会话恢复。在此基础上，聊天详情页面自身的局部状态管理机制（通常为页面级的数据对象）则专注于管理该页面内部的临时状态及用户输入，例如当前输入框中的文本内容。当用户成功登录后，其认证信息会同步更新至全局状态容器和本地缓存，确保各模块间状态的一致性。

聊天详情页的核心状态是维护消息列表的数组，其内容的动态更新（如新增消息、加载历史消息）通过调用前端框架提供的标准机制（例如通过特定函数修改页面数据并触发界面自动重新渲染）来实现。此外，各个会话的未读消息数量作为一项重要的用户提示信息，其状态被存储于全局状态容器中；当新的未读消息到达时，系统会相应更新此全局状态，从而确保在应用内各相关界面（如“消息”主页的会话列表）都能准确、即时地反映未读消息的总数。

2.3.3 医患随访模块

(1) 功能概述

本模块是构建并维系患者与医生之间长期、有效互动关系的功能单元，其核心业务逻辑聚焦于实现患者发起绑定申请、医生进行审批管理以及双方持续维护已建立的绑定状态。医患双方一旦成功确立绑定关系，系统将相应地解锁患者访问其绑定医生特定联系信息的权限，同时更为关键地，授权医生查阅该患者授权范围内的特定健康数据（例如，患者的日常健康记录、血糖变化趋势图表等），并正式开通医患双方间的加密实时通信功能。从前端的用户界面组织来看，相关功能被合理地分布于多个页面：例如，“我的医生”界面，作为患者端功能的一个按需加载的扩展模块，是患者查看已绑定医生列表、追踪申请状态以及发起新的绑定申请的主要入口；而“添加医生”界面，则作为“我的医生”功能下的子目录，专供患者依据条件搜索目标医生并发送绑定请求。在医生端，“我的患者”界面通常整合于其核心功能模块内，作为医生高效管理其患者列表、集中审批患

者绑定申请的核心工作区域。值得一提的是，医生端的“我的患者”界面在设计与实现上，复用了患者端“我的医生”界面的部分底层组件及核心交互逻辑，通过传入不同参数以适配医生角色的特定需求，体现了组件化开发的思想。

(2) 交互流程

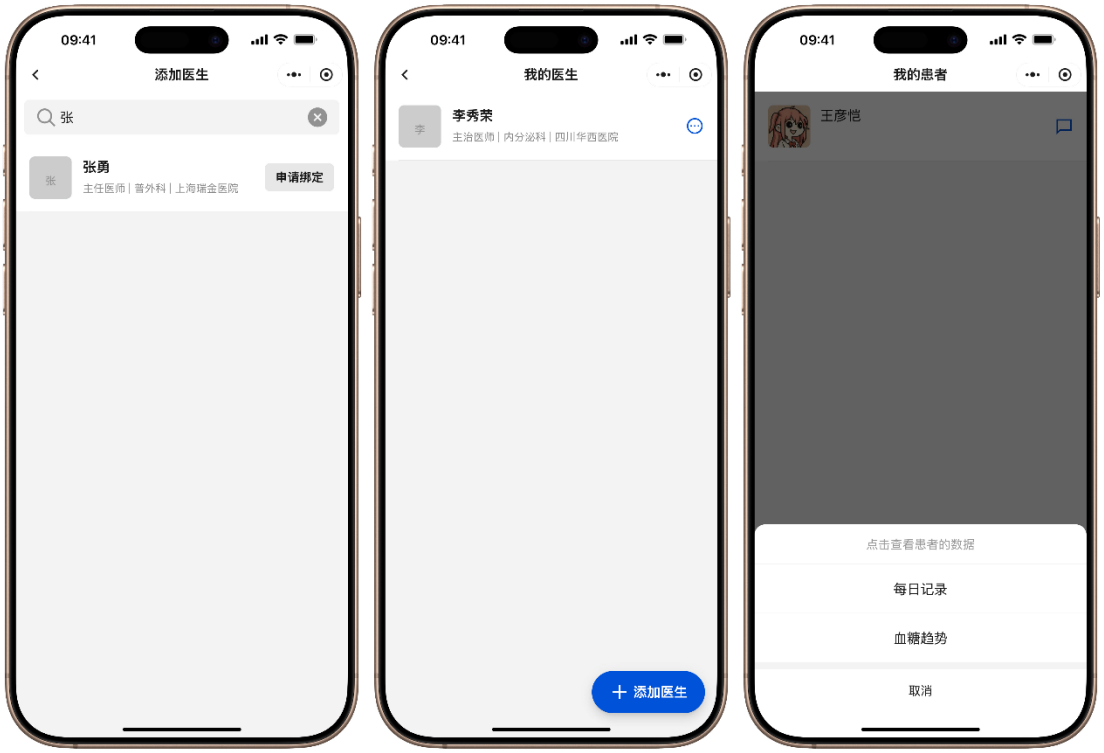
本模块的用户交互流程依据“患者”与“医生”这两种核心角色进行了清晰的区分设计。在患者端，用户通常从应用首页的相关入口导航至“我的医生”界面。该界面加载时，会通过与后端服务通讯，动态获取并展示当前用户已成功绑定或正在申请中的医生列表。每个医生信息卡片上均提供了便捷的操作入口，例如允许用户直接导航至与该医生的加密聊天界面。患者可在“我的医生”界面中便捷地找到“添加医生”的功能入口，从而进入医生搜索页面，在此页面依据姓名、科室等信息查找目标医生，并向其发送建立随访关系的绑定申请。在医生列表中，对于已发送申请但医生尚未处理的条目，系统支持通过标准的用户交互方式（如滑动操作唤出上下文菜单）显示“取消申请”的操作选项；对于已成功绑定的医生，则同样支持显示“解除绑定”的操作。

在医生端，用户登录后通常可直接在其工作台（如首页内嵌的“我的患者”视图）查看到已绑定及待审批的患者列表。对于处于“申请中”状态的患者卡片，系统会清晰地提供“同意”或“拒绝”的操作按钮，医生点击后将触发与后端服务的通讯，以完成对该绑定请求的处理。对于已成功绑定的患者，医生通过点击相应的患者卡片，即可导航至该患者的详细信息查阅界面，在此界面中可查看其日常健康记录、血糖变化趋势图，或直接发起实时聊天。这些供医生查阅的患者详细数据界面，在设计上复用了患者端原有的相关页面组件与布局，通过传递目标患者的唯一标识参数，由后端服务精确返回该特定患者的数据供医生查阅，确保了数据展示的准确性与界面风格的一致性。

功能解锁与状态精确控制是本模块交互设计的核心要素。诸如实时聊天、查阅对方健康数据等核心随访功能的使用权限，均与医患双方的绑定状态进行严格的动态关联。只有在双方成功建立绑定关系后，前端界面才会相应地解除这些高级功能的使用限制（例如，相关操作按钮由禁用状态转为可点击状态，对应的功能链接变为可见）。界面上的按钮文本、信息卡片的视觉状态（如高亮、角标）等，均会依据当前的医患绑定状态（如“未绑定”、“申请中”、“已绑定”、“已拒

绝”等）以及当前操作用户的角色（患者或医生），进行实时的动态调整与显示。这种设计不仅确保了用户界面能准确反映当前的业务状态，并能据此提供恰当的操作选项，还有效提高了系统的整体直观性与用户操作的便捷性。

此模块的页面展示如图 2.5 所示。分别展示了三个页面，为“添加医生”页，“我的医生”页与“我的患者”页。



（a）　　（b）　　（c）

图 2.5 医患随访模块。（a）“添加医生”页，（b）“我的医生”页，（c）“我的患者”页。

(3) 状态管理

医患随访模块的状态管理严格遵循角色职责分明的设计原则：患者作为主动方发起绑定请求，医生则拥有最终的审批决策权限，并且前端用户界面会清晰、准确地体现出发起者与审批者在操作权限与界面展示上的差异。系统的所有权限控制逻辑均由后端服务严格执行与保障，前端应用仅依据后端返回的状态信息和用户角色信息来动态适配界面的内容展示与可执行交互。本模块广泛采用了状态驱动型的界面设计范式，前端界面中的各类元素（如列表、按钮、文本提示等）及其可操作性，高度依赖于医患双方实时的“绑定状态”进行动态渲染与控制。

列表内容的构成、各个操作按钮的可见性与可用性、以及可执行的具体操作类型等，均依据“未绑定”、“申请中”或“已绑定”等不同的业务状态进行精确管理，此举不仅简化了前端的条件判断逻辑，也有效确保了用户界面与后端数据状态的实时同步。

在 UI 的交互设计上，本模块力求在满足角色功能差异化的同时，保持操作模式的一致性：例如，患者和医生在处理“申请中”状态的绑定请求时，所执行的操作是不同的（患者可选择“取消申请”，而医生则可选择“同意”或“拒绝”），因此界面上显示的按钮文本也会有所区别；然而，诸如通过列表项滑动操作来显示相关上下文操作按钮的交互模式，则在患者端与医生端保持统一。这种兼顾差异化需求与操作一致性的设计，既满足了不同用户角色的独特业务需求，也降低了用户的学习成本，提升了系统的整体易用性。此外，系统中还充分体现了功能模块与 UI 组件复用的设计思想：一个显著的例子是，当医生需要查看某一患者的健康记录或血糖趋势时，前端应用会复用患者端原有的相应数据显示页面组件，仅通过传递目标患者的唯一标识（Identification, ID）参数来动态获取并展示该患者的数据。这种复用策略不仅显著提高了开发效率，也增强了代码库的可维护性与一致性。通过在前端将列表滑动检测及动态操作按钮显示等通用交互逻辑封装于可复用的 UI 组件内部，确保了患者列表与医生列表在操作视觉行为上的一致性。

2.3.4 数据展示与录入模块

(1) 功能概述

本模块在系统中承担着两大核心职责：其一为多维度健康数据的便捷高效录入，其二为历史数据的结构化分类展示与查阅。在数据录入方面，系统特别设计了统一的表单界面，允许用户在一次操作中便捷地提交血糖、胰岛素、饮食等多项关联健康数据，此设计强调了数据录入的即时性、便捷性与各项生理指标间的内在关联性。在数据展示方面，系统能够将用户累积的历史记录按照数据类型进行清晰分类，并提供包括条件筛选、多维度排序及分页加载等在内的高级查阅与管理能力。从前端界面组织来看，这两大功能主要整合于某一按需加载的功能扩展模块（例如，从首页功能区进入的特定模块）下的“数据展示”界面与“数据

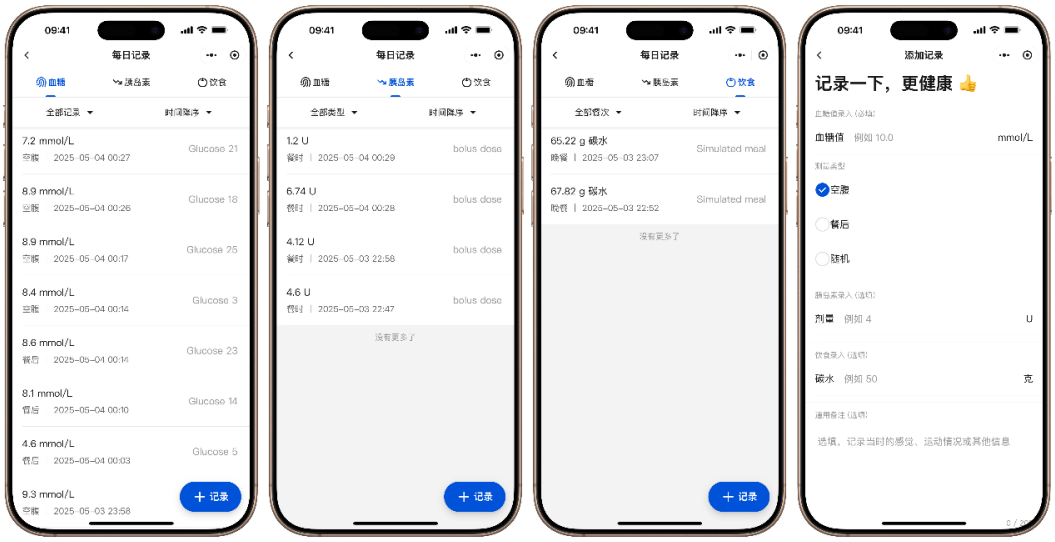
记录”界面，从而在逻辑上清晰地分离了用户“写入数据”与“读取数据”的操作场景。值得注意的是，“数据展示”界面在设计层面充分考虑了其在不同用户角色间的复用性：该界面组件能够接收用户 ID 作为参数，当医生角色用户访问时，可据此请求并展示特定患者的健康数据记录；而当患者角色用户自身访问时，则默认返回并展示当前登录用户的个人记录。此种设计体现了良好的模块化与组件复用思想，有效提升了开发效率与系统维护性。

(2) 交互流程

“数据展示”界面的核心交互采用了常见的页签（Tab）式结构组织，顶部通常设有“血糖记录”、“胰岛素记录”、“饮食记录”等多个页签，用户通过点选不同的页签即可切换至相应数据类型的历史记录列表。每个数据列表均支持用户所熟悉的移动端标准交互模式，例如通过下拉操作刷新当前列表数据，以及在列表滚动至底部时自动加载更多历史记录，这些设计旨在提升数据浏览的效率与用户体验。为便于用户从大量记录中快速定位目标信息，各个数据列表均集成了多条件筛选功能（例如，可按血糖测量时点、胰岛素具体类型、饮食餐次等进行筛选）和灵活的排序机制（例如，可按记录时间先后或数值高低进行排序）。此外，在该界面的右下角通常设有一个视觉上较为醒目的悬浮操作按钮，用户点击此按钮即可快速导航至“数据记录”界面，进行新的健康数据录入。

“数据记录”界面则提供了一个一体化的数据提交表单，允许用户在同一界面一次性输入包括血糖值、胰岛素使用详情（如类型、剂量）、饮食内容概要（如碳水化合物估值）及相关备注信息在内的多项数据。该表单的提交逻辑依据用户实际填写情况动态决定最终提交的数据字段组合：例如，若用户仅填写了血糖相关的备注信息，则提交的数据将主要包含血糖相关的字段；若用户填写了胰岛素剂量，则系统会要求必须同时选择胰岛素类型方可成功提交。数据成功提交后，系统会给予用户一个简短的即时反馈（例如，通过短暂的成功提示消息框），并通常会自动将用户导航回上一级界面（多数情况下是“数据展示”界面），同时触发该界面对应数据页签的列表刷新机制，以确保用户能够立即查看到刚刚提交的新记录。为提升用户记录数据的积极性与依从性，每次进入数据录入界面时，系统还会随机展示一条与积极健康管理相关的鼓励性短语，以此对用户进行正向激励。

此模块的页面展示如图 2.6 所示。分别展示了四个页面，为“数据展示”页中的血糖、胰岛素与碳水展示，以及“添加记录”页。



（a） （b） （c） （d）

图 2.6 数据展示与录入模块。（a）血糖展示，（b）胰岛素展示，（c）碳水展示，（d）“添加记录”页。

(3) 状态管理

本模块在状态管理层面体现了若干精心设计的策略。首先，针对多类型健康数据的统一管理，系统采用了基于配置化的实现方式。前端的“数据展示”界面通过一个结构化的配置对象（例如，一个包含各记录类型详细配置的数组）来统一管理不同记录类型（血糖、胰岛素、饮食等）的展示逻辑与交互行为。此配置对象中详细定义了每种记录类型对应的后端 API 路径、可用的筛选选项、支持的排序方式、数据字段与显示标签的映射关系，乃至单个记录卡片的具体渲染逻辑等。通过这种方式，不同类型的健康记录能够被统一地映射到一个标准化的卡片结构上进行展示，利用通用的显示字段（例如，用于展示摘要信息、记录时间、备注内容等的标准化字段）来保持界面布局的视觉一致性与用户体验的连贯性。

其次，在数据提交方面，系统实现了智能化的单位转换与条件提交逻辑。数据录入界面的提交处理逻辑具备一定的智能性，它会根据用户在表单中实际填写的字段，有条件地在最终提交给后端的数据结构中包含血糖、胰岛素、饮食等相应的子结构体，确保仅提交用户实际填写的有效数据（例如，胰岛素记录必须同时包含剂量和类型信息才被视为有效并提交）。此外，系统还支持关键数据（如

血糖值）的单位自动转换功能：前端集成了一个专用的血糖格式转换工具或函数，能够将用户习惯输入的血糖单位（例如 mmol/L）自动转换为后端系统统一采用的存储单位（例如 mg/dL）后再进行提交。这种设计不仅降低了用户的录入负担，避免了因单位混淆可能导致的数据错误，也实现了前端显示单位与后端存储单位的标准化与解耦，形成了前端统一使用 mmol/L、后端统一使用 mg/dL 的良好数据管理实践。所有涉及时间戳的信息，在前后端传输及存储过程中，均统一采用国际标准化组织（International Organization for Standardization, UIISO）制定的 ISO 8601 日期和时间表示法，以保证时间信息的精确性、一致性与跨系统兼容性。

2.3.5 数据可视化展示模块

(1) 功能概述

该模块的核心使命在于将离散、时序性的血糖记录数据，转化为用户易于理解且具有洞察力的视觉化健康图景。其核心功能是通过交互式的图表组件，清晰、直观地呈现用户血糖水平在不同时间维度下的动态变化趋势与模式。此种可视化表达不仅有助于患者更深入地理解自身血糖波动的内在规律与潜在影响因素，亦为临床医生远程识别潜在的高血糖或低血糖风险、客观评估治疗方案的实际效果以及辅助制定更具个体针对性的管理策略，提供了至关重要的数据依据与决策支持。本模块支持用户灵活选择不同的时间粒度（例如，日视图、周视图、月视图乃至更长的时间周期）来审视血糖模式，从而充分满足患者日常自我监测精细化管理及医生远程随访宏观调控的双重需求。前端的实现主要整合于某一按需加载的功能扩展模块（例如，从首页功能区进入的特定模块）下的“血糖趋势”专属界面，并集成了成熟的客户端图表渲染库（如基于 JavaScript 的 ECharts 等）以高效、美观地绘制各类统计图表。

(2) 交互流程

“血糖趋势”界面的整体设计力求直观、简洁且易于操作。界面顶部通常设有一个清晰的页签（Tab）切换栏，为用户提供了包括“日”、“周”、“月”、“6 个月”、“年”在内的共五种预设时间维度视图，供用户根据需要自由选择。当页面首次加载时，系统默认展示“日”维度（即当日）的血糖数据图表。用户通过点选不同的时间维度页签，系统会自动触发对应时间跨度与粒度的血糖数据加载请求，并实时刷新图表内容。图表组件在渲染时采用了按需加载或延迟渲染的策略，

即仅在图表组件实际进入用户可视区域且页面逻辑准备就绪时才进行初始化与绘制，此举有效避免了不必要的初始计算资源消耗，显著优化了页面的整体加载速度与响应性能。一旦图表实例成功获取并初始化后，后续所有的数据更新与视图调整均通过调用该图表库提供的标准实例方法来动态完成，从而提高了在不同时间维度数据间切换时的流畅性与用户体验。

为增强信息的可读性与上下文的清晰度，图表区域的上方会自动显示当前图表所涵盖的具体日期或时间范围，作为动态生成的标题。此外，为了提升用户与图表的交互性及数据的可读性，当用户在生成的折线图上点击或悬停于某一特定数据点时，图表库内置的提示框（Tooltip）功能会被激活，清晰展示该时间点的具体血糖数值、测量时间等详细信息。

此模块的页面展示如图 2.7 所示，即为“血糖趋势”页。

图 2.7 数据可视化模块。“血糖趋势”页。

(3) 状态管理

本模块在状态管理层面展现出若干关键特性。首先是响应式的图表渲染机制

与高效的延迟加载策略：图表组件利用延迟渲染模式，有效减少了页面的初始加载时间与资源占用。前端在获得图表库的实例对象后，所有后续的数据更新、重绘等操作均直接作用于此已初始化的实例，实现了高效的动态数据更新和流畅的界面渲染。同时，所选用的图表库通常会自动侦测并适配不同用户设备的屏幕像素密度，以确保图表在各类高分辨率或普通分辨率设备上均能保持清晰、锐利的显示效果。

其次，是多时间维度切换的灵活性与后端数据聚合的智能性：系统支持用户在预设的五种时间维度间自由切换，每种维度对应着不同的数据聚合粒度（例如，“日”视图可能直接展示原始的血糖监测数据点，而“月”视图或“年”视图则可能展示经过后端聚合计算后的日平均值或特定统计指标）。此设计的关键在于，复杂或计算密集型的数据聚合任务主要由后端服务承担完成；前端应用仅需通过 API 接口参数告知后端所需的数据时间维度、起止范围等条件，后端便会返回已经预处理完毕、可直接用于图表绘制的结构化数据集。这种职责划分极大地减轻了前端客户端的数据处理负担，显著提升了图表数据加载及不同维度间切换的速度与效率。

此外，该界面的设计还充分考虑了其在不同用户角色间的复用性。通过在 API 请求中传递当前操作用户的唯一标识（user_id）参数，同一套“血糖趋势”界面逻辑既可服务于患者本人查看自身数据，亦可供医生在获得授权后查看其名下特定患者的血糖趋势图，无需为不同角色开发独立的界面，体现了良好的设计复用性。最后，是简洁高效的 API 交互模式：其核心交互主要围绕一个专门设计的 API 端点进行（例如，一个通过 HTTP GET 方法访问的 /glucose/trend 类似路径的接口）。前端通过动态改变请求中的参数组合（如代表时间维度的标识、具体的日期范围参数，以及可选的患者用户 ID 参数）来获取不同维度或不同患者的血糖趋势数据。后端返回的数据结构经过精心设计与精简，通常直接提供了图表绘制所需的横坐标轴标签序列、主要的血糖数据序列，以及描述当前数据覆盖时间范围的文本等关键字段，使得前端应用能够快速、直接地将这些数据应用于图表库的渲染配置中，进一步提升了整体的交互效率。

2.3.6 血糖预测结果展示模块

(1) 功能概述

本模块是系统深度融合人工智能技术、为用户提供前瞻性健康洞察的核心功能体现。其核心职责在于接收并以清晰、易懂且负责任的方式，向用户呈现由后端深度学习模型生成的对患者未来短期血糖水平的预测结果。通过提供未来特定时间点（例如 30 分钟或 60 分钟后）的血糖预测值，本模块旨在辅助患者提前预判潜在的高血糖或低血糖风险，从而为其及时调整饮食、运动等日常行为以进行主动干预，提供有价值的参考信息。在设计上，本模块高度强调医疗预测信息传达的审慎性，通过在界面显著位置设置清晰的免责声明，提示用户所有预测结果仅供参考，绝不能替代专业的医疗诊断与建议。前端的实现主要由某一按需加载的功能扩展模块（例如，从首页功能区进入的特定模块）下的“血糖预测”专属界面承载，该界面作为用户与预测功能直接交互的窗口，负责收集用户的预测意向参数、发起对后端预测服务的请求，并最终处理及展示返回的预测结果。

(2) 交互流程

“血糖预测”界面的用户交互设计力求简洁明了，所有核心界面元素均围绕预测操作的执行与预测结果的清晰展示进行布局。当页面加载完成时，其核心区域会展示一个用于选择预测时间区间的控件（例如，一组单选按钮或一个下拉选择菜单），通常提供“预测 30 分钟后血糖”与“预测 60 分钟后血糖”等若干预设选项。用户在选定期望的预测时长后，可通过点击界面上明确标识的“开始预测”按钮来触发整个预测流程。一旦用户点击该按钮，前端应用会即刻向后端预测服务发送一个包含所选预测时长（例如 30 分钟或 60 分钟）等必要参数的请求。在等待后端模型进行推理计算并返回响应的期间，该预测界面会显示相应的加载状态指示（例如，一个动态的加载动画），同时，“开始预测”按钮通常会被置为禁用状态，以防止用户因不耐或误操作而发起重复的预测请求。

后端模型完成推理计算后，一般会返回一个包含未来一系列连续时间点（通常以固定的较小时间间隔，如每 5 分钟为一个节点）的血糖预测值，即构成一个时间序列形态的预测数据集。前端应用在接收到这些数据后，会根据用户最初选择的预测时长，从中精确提取出用户最为关心的目标时刻（即 30 分钟或 60 分钟后）的血糖预测值。该数值在展示给用户前，会经过必要的单位转换处理（例如，从后端模型常用的 mg/dL 单位转换为前端界面统一采用且用户更为熟悉的 mmol/L 单位），并以醒目、直观的方式呈现在页面的显著信息区域。预测数值的

显示通常会保留小数点后一位，以兼顾医学计量的严谨性与普通用户的可读性。考虑到任何预测模型均存在固有的不确定性与局限性，在该页面的顶部或用户易于注意到的其他位置，会常驻一条重要的、可通过滚动或动画吸引用户注意的免责声明，其内容大意为："本预测结果仅供参考，不能替代专业医疗建议。若遇紧急健康状况，请务必及时就医。"该声明区域通常还会附带一个允许用户操作的"关闭"或"隐藏"按钮，以便用户在充分阅读并理解后可选择暂时将其隐藏，从而获得更佳的界面视野。

此模块的页面展示如图 2.8 所示，即为"血糖预测"页。

图 2.8 血糖预测结果模块。"血糖预测"页。

(3) 状态管理

本模块在状态管理方面，针对血糖预测结果的特殊性与用户体验需求，体现了若干具有针对性的设计特点。首先是血糖值单位的自动转换与显示格式的标准化。为适配后端模型输出数据可能采用的单位（如 mg/dL），而前端界面需统一向用户展示其更为熟悉的单位（如 mmol/L），前端在数据处理流程中引入了可靠的单位转换逻辑。此逻辑通常由一个专用的血糖单位换算工具或函数模块负责，

确保所有展示给用户的血糖预测值均符合预设的单位与格式标准。

其次，后端血糖预测模型通常会返回覆盖未来一段时间的、由多个数据点构成的时间序列预测数组。然而，为避免向用户呈现过量、可能引起困扰的复杂数据，并简化前端界面的处理逻辑与信息层级，前端应用在接收到完整的预测序列后，仅会从中提取并展示用户在发起预测时所明确选择的目标时刻（如 30 分钟后或 60 分钟后）的单一预测值。这种对信息的提炼与聚焦，旨在为用户提供最直接、最相关的决策参考。所展示的预测血糖值统一规范为保留一位小数，此举既符合医学领域对血糖数值精度的一般要求，也提升了界面信息的专业性与用户的可读性。

最后，关于目标预测时间点数值的提取逻辑，其设计兼顾了简洁性与高效性。鉴于后端模型通常以固定的时间粒度（例如，每 5 分钟）返回预测序列数组，前端应用仅需知晓用户请求的总预测时长（即预测期程，如 predict_minutes 参数所指定的值）以及此固定的时间粒度，即可通过简单的数组索引计算（例如，目标索引 = 预测期程 / 时间粒度 - 1，若数组从 0 开始计数），准确、高效地从返回的序列中定位并提取出目标时刻所对应的血糖预测值。这种设计使得前端的提取逻辑保持高度简洁且易于维护，同时亦能灵活支持未来可能对不同预测时长选项的调整或扩展，后端预测服务接口亦因此得以保持其通用性与稳定性。

2.4 代码质量分析

为确保前端代码的稳定性、可维护性和性能表现，还对代码库进行了全面的静态质量分析。检查范围涵盖代码风格规范、潜在错误检测、以及针对微信小程序特性的性能优化项（如包体积、资源利用）。如图 2.9 所示，分析结果表明所有检查项均已通过，具体体现为：主包体积严格控制在 1.5MB 以下；无未使用的代码和冗余资源；代码文件经过充分压缩；组件按需加载；且代码中不含敏感信息。这些质量控制措施为系统构建了坚实的技术基础。

2.5 本章小结

本章系统地介绍了前端微信小程序的整体设计与核心功能实现。从基础技术选型到架构宏观布局，再到各主要业务模块的具体实现策略，详细探讨了前端如何构建直观用户界面、管理复杂状态并与后端高效交互。前端设计充分利用了微

信小程序平台的特性，旨在提供流畅、安全且便捷的用户体验。

需要指出的是，前端是系统的表现层，所有业务逻辑处理、数据存储及核心

图 2.9 代码质量分析

计算（如 AI 模型）均依赖于后端服务和数据库系统的支撑。下一章将深入探讨本系统的后端架构设计、主要服务实现以及数据库选型与模式设计，揭示系统如何在后端构建稳定可靠的数据处理和业务支撑能力。

第三章 后端与数据库设计与实现

后端服务与数据库是支撑本系统稳定运行和核心功能实现的基石。它们负责处理前端请求、执行业务逻辑、管理关键数据并支持智能模块。本章将详细阐述后端的技术选型、整体架构设计以及 MySQL 与 MongoDB 混合数据库的设计与实现。最后，将介绍几个核心后端模块的具体实现方式。

3.1 后端技术选型与概述

系统后端在整个应用架构中扮演着核心角色，它负责连接前端用户界面、执行复杂的业务逻辑、管理数据的持久化存储，以及集成关键的人工智能模块。因此，后端的技术选型对于确保系统的整体性能、可扩展性、可维护性以及开发效率至关重要。在本系统中，后端主要基于 Python 语言及 FastAPI 框架构建，并结合使用了 MySQL 和 MongoDB 两种数据库进行数据存储。

选择 Python 作为主要的后端开发语言，是基于其在多个方面展现出的显著优势。首先，Python 拥有简洁易读的语法，这使得代码编写和维护变得更为高效，同时也降低了团队协作的门槛。其次，Python 最突出的优势在于其庞大且活跃的生态系统，特别是在数据科学和人工智能领域。丰富的第三方库，如用于数据处理的 Pandas、数值计算的 NumPy，以及机器学习框架 Scikit-learn、TensorFlow、PyTorch 等，为本系统后续集成血糖预测等 AI 功能提供了极大的便利和强大的支持，这与项目的需求高度契合。此外，Python 社区的良好支持提供了大量的文档、教程和问题解决方案，有助于开发者快速克服遇到的技术挑战。Python 的跨平台性也增强了系统的部署灵活性，使其能够在多种操作系统环境下稳定运行。

在 Web 框架的选择上，采用了 FastAPI，这是一个现代化的 Web 框架，专门用于高效地构建 API。其选择主要基于其卓越的性能和多项核心特性。FastAPI 构建于轻量级、高性能的 ASGI 框架和优秀的数据校验与序列化之上，使其成为 Python Web 框架中性能最优者之一，能够有效处理高并发请求，满足实时聊天和数据传输等需求。FastAPI 对异步编程的原生支持是另一大优势。这使得后端能够以非阻塞的方式高效处理 I/O 密集型任务，例如异步数据库访问和调用外部服务，从而显著提升系统的响应速度和吞吐量。利用 Pydantic 库，FastAPI 能够基于 Python 类型提示自动进行请求数据的校验和响应数据的序列化，这极大地减

少了手动编写校验和转换代码的工作量，提高了开发效率并保障了数据结构的准确性。FastAPI 还能根据代码自动生成符合 OpenAPI (Swagger UI) 和 ReDoc 标准的交互式 API 文档，极大地便利了前后端协作和 API 的维护。其强大的依赖注入系统使得组织代码逻辑、管理数据库连接和处理认证等横切关注点变得更为简洁高效，有助于构建模块化和可测试的应用。这些特性使 FastAPI 非常适合构建本系统所需的 RESTful API 服务和 WebSocket 服务，提供了高性能、高效率和良好的开发体验。

除了核心的 Python 语言和 FastAPI 框架，本系统后端技术栈还集成了其他关键技术组件，共同构成了完整的服务能力。数据存储方面，我们结合使用了 MySQL 和 MongoDB。MySQL 作为关系型数据库，适用于存储结构化、关系性强的数据；而 MongoDB 作为文档型数据库，则适用于存储灵活、写入频繁的非结构化或半结构化数据，充分利用了各自的优势。与这些数据库交互时，采用了 Python 中成熟的异步驱动库。用户身份认证采用了业界标准的 JWT 机制，实现安全的无状态认证。为了支持实时聊天等功能，系统集成了 WebSocket 服务，实现了前端与后端之间的实时双向通信。此外，后端还负责集成外部服务，例如调用微信 API 获取用户 OpenID，以及与人工智能大模型 API 进行交互以支持智能咨询功能。

3.2 后端系统架构设计

本系统的后端设计采用了经典的分层架构思想，并结合 FastAPI 框架的特性进行优化，以实现模块化、高内聚、低耦合的目标。这种架构模式有利于分离关注点，提升代码的可维护性、可测试性以及团队协作效率。

后端系统按照功能职责被逻辑地划分为以下几个主要层次和模块：

（1）API 层：作为系统的入口，负责接收前端或其他客户端的请求。该层定义 HTTP RESTful API 端点和 WebSocket 通信端点，利用 Pydantic 模型对请求数据进行自动化校验，并调用依赖注入层和服务层处理请求，最终将处理结果格式化为标准响应返回给客户端。此层保持轻量，专注于请求的接收、路由分发和响应封装。

（2）依赖注入层：负责管理并提供跨越多个 API 端点或服务组件所需的通

用功能和资源。通过利用 FastAPI 的依赖注入机制，它能够处理用户身份认证、Token 校验，以及获取和管理数据库连接或会话实例。通过依赖注入，上层模块无需关心资源的创建和生命周期管理，增强了模块的独立性。

（3）服务层：这是系统的核心业务逻辑层，负责实现具体的业务流程和复杂的计算。该层包含了用户认证、医患随访、聊天消息、健康数据管理、AI 预测等模块的业务逻辑。服务层协调与数据访问层的交互，执行数据库操作，并调用工具层提供的通用功能。服务层内部的服务之间也可相互协作，是业务规则的执行者，与 API 层和底层数据存储细节解耦。

（4）数据访问层：此层专门负责与具体的数据库系统进行交互。它包含数据库连接配置和管理逻辑，并定义数据结构的抽象模型（如 SQLAlchemy ORM 模型）。数据访问层对上层服务层透明化了底层数据库驱动和连接管理的细节，确保业务逻辑层不必关注数据库的具体实现。

（5）数据结构模型层：集中使用 Pydantic 定义系统中的各种数据结构模型。这些模型用于验证 API 请求数据，序列化 API 响应数据为 JSON 格式，以及在服务层之间传递结构化数据，作为数据契约的定义。

（6）工具层：提供一系列可重用的通用功能函数和辅助类，为上层服务层提供基础支持。其职责涵盖安全相关功能（如 JWT）、时间处理、外部服务 API 调用封装（如微信 API 或大模型 API）、以及 AI 预测所需的数据处理等，提高了代码的复用性。

（7）核心配置层：负责管理系统的全局配置设置、日志记录配置等基础设施相关任务，使得系统各项设置易于调整。

整个后端系统通过 FastAPI 的应用实例将上述各个组件组织起来，进行路由注册、加载配置、并构建依赖关系，协同对外提供服务。

本系统的 HTTP API 设计遵循 RESTful 架构风格，旨在提供一致性、无状态且易于理解的接口集合。API 端点将系统实体视为资源，使用清晰、有语义的 URL 路径进行标识（如 /api/v1/users）。系统利用 HTTP 方法（GET, POST, PUT, DELETE）的语义表示对资源的操作。后端设计为无状态，用户认证通过请求头中的 JWT Token 实现。请求体和响应体采用 JSON 格式，通过 Pydantic 模型自动校验和序列化。后端返回标准的 HTTP 状态码反映请求处理结果，业务逻辑错误在响应体

中提供详细信息。所有 API 路径使用/api/v1/前缀进行版本控制，并基于 FastAPI 自动生成 OpenAPI 文档，便利了前后端协作。

后端系统需要高效协调内部组件以及与外部依赖（数据库、AI 模块）之间的协作。后端与数据库的协作是服务层的核心功能之一。服务层通过依赖注入获取数据访问层提供的异步数据库连接或会话，并利用 ORM/驱动方法执行 CRUD 操作。本系统采用混合数据库策略，将结构化数据存储在 MySQL，高频时间序列数据存储在 MongoDB，利用各自优势。不同数据库的数据通过服务层在应用内存中进行关联和整合。后端与 AI 模块的协作体现在将人工智能能力集成到后端架构中。预测流程由 prediction_service 管理，该服务从数据库获取历史数据，调用底层的预测逻辑执行推理，并处理预测结果。智能聊天功能由 chat_service 负责，当用户消息发送给智能助手时，chat_service 调用 llm_service 与外部大模型 API 进行交互，并将回复通过 WebSocket 推送给前端。这种分层和封装确保了核心业务逻辑与具体的 AI 实现细节解耦。

WebSocket 通信服务作为系统实时聊天功能的支撑，整合到 FastAPI 应用中。通过定义 WebSocket 端点，FastAPI 管理连接。连接处理函数利用依赖注入系统获取用户认证和数据库访问权限。WebSocket 消息的接收、处理和转发逻辑主要由 chat_service 在服务层进行业务逻辑编排，通过维护活跃连接列表实现消息的精确投递。

3.3 数据库设计与实现

系统的数据库设计采用了关系型数据库 MySQL 和文档型数据库 MongoDB 相结合的混合模式。这种混合架构旨在充分发挥两种数据库各自的技术优势，以最优化地满足系统中不同类型数据的存储和管理需求。

3.3.1 MySQL 数据存储设计

MySQL 作为一种成熟的关系型数据库管理系统，凭借其强大的事务支持、数据完整性约束、标准化的 SQL 查询语言以及在处理结构化数据方面的长期优势，被选用于存储系统中结构稳定、关系性强且需要严格数据一致性的核心业务数据。

主要存储在 MySQL 中的数据实体包括用户信息，存储所有用户的基本属性

如用户 ID（作为主键和唯一标识）、昵称、手机号、用户角色（患者或医生）和 OpenID 等。用户档案信息则分别存储在患者档案表（patient_profile）和医生档案表（doctor_profile）中，这些表通过外键与用户表（users）的用户 ID 相关联，确保每个档案都明确归属于一个具体用户，并记录患者的诊断日期、身高体重等或医生的职称、科室医院等专属信息。此外，医患绑定关系也存储在独立的绑定表（bindings）中，记录患者用户 ID 和医生用户 ID，同样通过外键关联至用户表，并维护绑定状态（如申请中、已接受、已拒绝、已解除）和相关时间戳。

在 MySQL 中，这些数据实体被组织成规范化的二维表结构，并通过外键约束来维护表与表之间的引用完整性，特别是与用户表的关系，以此保障数据的关联性和全局一致性。后端在进行数据访问和对象映射时，利用了 SQLAlchemy 异步 ORM 库，简化了与数据库的交互代码并支持异步操作。

3.3.2 MongoDB 数据存储设计

MongoDB 作为一种高性能、无模式的文档型数据库，以其灵活的数据结构、便捷的水平扩展能力以及对大量写操作的优异支持而著称，被选用于存储系统中那些数据结构可能随时间变化、写入频率较高或天然适合文档模型组织的非结构化或半结构化数据。

主要存储在 MongoDB 中的数据实体包括患者的日常健康记录，如血糖测量（blood_glucose）、胰岛素注射（insulin_records）和饮食摄入（diet_records）记录。这些记录产生频繁，且其内部结构可能因用户或具体的记录事件而略有差异（例如，饮食记录可能只包含碳水化合物估算值或详细的食物描述），文档模型非常适合存储单个时间点或事件的完整、自包含记录。每条记录文档中都包含一个 user_id 字段，用于将其关联到 MySQL 中存储的用户信息。聊天消息（chat_messages）也是存储在 MongoDB 中的一个重要数据实体，用于存储医患之间或患者与智能助手之间的实时交流内容。聊天消息的数据结构包含了发送者 ID、接收者 ID、消息内容、时间戳、消息类型、已读状态以及是否为 AI 消息等信息，天然适合存储为独立的文档。通过 chatId 字段（通常由会话参与者用户 ID 的组合生成）和 user_id 字段，实现了消息与特定会话和用户的关联。

在 MongoDB 中，这些数据被存储在各自独立的 Collection 中，每条记录是一个独立的 JSON 格式文档。虽然文档结构提供了灵活性，但在核心字段（如

user_id, timestamp）上保持了一致性，以便于进行查询和关联操作。后端通过 Motor 异步驱动库进行数据访问，直接操作文档，充分利用了 MongoDB 的高性能特性。

3.3.3 混合数据库架构与协同策略

本系统采用混合数据库架构的核心理由在于能够充分利用关系型数据库和文档型数据库各自的技术优势，以最佳方式适配系统中不同类型数据的存储和管理需求。这使得系统能够利用 MySQL 的事务和强一致性特性来保障核心关系数据的完整性与可靠性，同时利用 MongoDB 模式的灵活性和高性能写入能力来高效处理高频产生的、结构可能变化的时间序列数据和非结构化数据。这种分而治之的策略将不同类型的数据存储在最适合它们的数据库中，避免了在单一数据库中处理所有数据类型可能带来的复杂性或性能瓶颈，实现了数据存储的有效解耦。

在混合数据库架构下，关键的协同策略体现在应用层面的数据关联。系统中存储在 MongoDB 的大多数文档（如健康记录、聊天消息）都包含一个指向 MySQL 中用户表（users）的 user_id 字段。这种设计允许后端服务层在处理业务请求时，根据用户 ID 从 MySQL 中快速获取用户的基本信息或档案信息，并同时从 MongoDB 中查询该用户相关的健康记录列表、血糖趋势数据或聊天历史记录。服务层随后在应用内存中将这些来自不同数据库的数据进行整合或关联展示给用户。

例如，在生成聊天摘要列表时，后端服务会先从 MongoDB 的 chat_messages 集合中聚合获取用户的最新消息和未读计数，然后提取出所有对话对方的 user_id 列表，再使用这些 ID 从 MySQL 的 users 表中批量查询对应的用户昵称和头像等基础信息，最后在服务层将这些信息进行组合，构建成完整的聊天会话摘要列表返回给前端。同样，医生查看患者健康数据时，后端服务需要从 MySQL 获取患者的档案信息，并从 MongoDB 查询其血糖记录等进行统计分析。这种协同方式不涉及数据库之间的数据同步复制，而是依赖于后端服务在处理具体业务请求时，根据业务逻辑逻辑性地进行跨库查询和数据组合。这种策略适用于两个数据库之间不存在频繁的、需要强一致性保障的实时数据同步场景，且主要关联点是通过用户 ID 进行的实体关联。

3.4 后端核心模块实现

本系统的后端服务承载了多项核心功能，为前端用户界面及各项复杂业务逻辑提供了稳定且高效的支撑。以下将分别阐述用户管理与认证、健康数据记录与查询、血糖预测服务以及实时通信这几个关键业务模块在后端的技术实现策略与核心逻辑。

3.4.1 用户管理与认证接口

用户管理与认证模块是整个系统安全访问与个性化服务的基础，其核心职责在于用户身份的建立、精确验证与持续维护。后端在此模块中实现了一系列关键功能，包括新用户的注册、既有用户的登录、用户基本信息的获取与更新等。这些功能的核心实现逻辑被内聚于专门的认证服务模块与用户服务模块之中，并通过一组遵循 RESTful 设计原则的 API 端点对外提供标准化的服务。

用户认证流程设计兼顾了安全性与便捷性，支持基于用户手机号码与动态验证码的传统认证方式，同时也兼容通过微信小程序平台提供的开放接口（例如，利用 Jscode 换取用户的 OpenID）进行授权的快捷登录模式。当用户发起注册或登录请求时，后端服务会依据其提供的手机号码或 OpenID，在关系型数据库（MySQL）中专门设计的用户核心数据表中查询相应的用户记录。若用户身份通过验证，后端将调用一个独立的安全工具模块中的函数，生成并签发 JWT。此 JWT 内嵌了用户的唯一身份标识（例如用户 ID），并使用安全的密钥进行数字签名，以保障其真实性与完整性。生成的 JWT 随后会在登录成功的响应中安全地返回给前端应用。

对于后续所有需要认证访问的 API 请求，前端应用均需在其 HTTP 请求头部中携带此 JWT。后端系统通过其 Web 框架（如 FastAPI）提供的依赖注入机制，在专门的认证依赖处理模块中定义了请求拦截与认证逻辑。该依赖处理模块会自动拦截所有受保护的 API 请求，从请求头中提取 JWT，并调用前述安全工具模块中的相应函数对令牌进行解码与严格验证，验证内容包括但不限于签名的有效性、令牌的签发者、以及是否在预设的有效期内。一旦令牌验证成功，该依赖处理模块还会根据令牌内含的用户 ID，从 MySQL 数据库中加载完整的用户对象信息（通常包括其关联的个人档案信息），并将此用户对象注入到 API 端点

处理函数的参数中，供后续的业务逻辑直接、安全地使用。针对 WebSocket 实时通信连接的认证需求，系统亦在相应的依赖管理模块中提供了类似的认证逻辑，从连接建立时的参数中获取并验证用户令牌。至于用户个人信息的查看与更新等操作，则由用户服务模块负责处理，该模块通过注入的 MySQL 异步数据库会话实例，对存储用户基础信息及详细档案信息的相关数据表执行高效的查询与更新操作。

3.4.2 健康数据记录与查询接口

健康数据记录与查询模块的核心职责在于安全、高效地接收、持久化存储并按需检索患者在日常生活中产生的各类健康相关数据，主要涵盖血糖测量值、胰岛素使用记录以及饮食摄入详情等。这部分数据构成了后续进行健康状态监测、个体化趋势分析以及智能预测建模不可或缺的基础。

数据的录入功能通过一个设计统一的 API 端点（例如，一个支持 POST 请求的 /record/add 类似路径的接口）来实现。该接口能够接收前端应用提交的、可能包含血糖、胰岛素和饮食等多种类型数据的组合式数据结构。后端服务在接收到此请求后，会首先对请求体进行解析与校验，然后依据所包含的数据子结构类型，将各类健康数据分别、准确地存储到文档型数据库（MongoDB）中预设的对应数据集合（Collection）内（例如，血糖记录集合、胰岛素使用记录集合、饮食记录集合等）。MongoDB 的文档数据模型因其结构灵活性与高写入性能，非常适合存储此类结构可能多样化且产生频率较高的时间序列型健康数据。

数据的查询功能则通过一系列设计精细的 API 端点（例如，分别用于获取血糖、胰岛素、饮食记录的“/glucose/getPagedRecords”、“/insulin/getPagedRecords”、“/diet/getPagedRecords”等类似路径，均支持 GET 请求）提供分页查询服务。这些查询接口通常支持传入分页参数（如页码、每页条数）、排序参数（如按时间或数值排序）以及针对不同数据类型定制的特定筛选条件（例如，血糖记录的测量标签、胰岛素的种类、饮食记录的餐次类型等）。后端服务在接收到查询请求后，会利用注入的 MongoDB 数据集合操作对象，构建相应的数据库查询语句，执行包括按用户 ID 精确过滤、按指定字段排序、以及分页限制等在内的复杂查询操作，最终从 MongoDB 中高效检索出符合条件的数据并返回给前端。特别地，对于医生角色用户查看其名下特定患者健康记录的场景，相关查询接口还会额外

接收一个代表目标患者的用户 ID 参数，后端服务在处理此类请求时，不仅会依据此参数筛选数据，还会在服务层进行严格的权限验证（例如，确认当前操作医生与目标患者之间存在有效的绑定随访关系），以确保数据访问的合规性与安全性。

3.4.3 血糖预测接口

血糖预测模块是本系统深度集成人工智能能力、实现健康管理前瞻性与智能化的核心体现。该模块负责接收来自前端的血糖预测请求，调度并执行后端部署的深度学习模型进行推理计算，最终将未来一段时间内的血糖预测结果返回给用户。此处的阐述主要聚焦于血糖预测功能在后端的技术实现框架，其具体的模型原理与算法细节将在第四章进行深入探讨。

血糖预测功能主要通过一个专门定义的 API 端点（例如，一个支持 POST 请求的“/prediction/predict_glucose”类似路径的接口）来触发。其核心的预测业务逻辑被封装在一个独立的血糖预测服务模块（prediction_service）中。当后端接收到前端发送的预测请求（通常包含如“期望预测未来多少分钟”的预测时长参数）时，该预测服务模块首先会调用另一个专门的患者数据服务模块（patient_data_service），从关系型数据库（MySQL）中获取当前用户的基本档案信息，并从文档型数据库（MongoDB）中查询其近期的血糖统计数据及相关健康记录。在某些高级预测策略中，预测服务模块还可能基于这些信息执行一个病人相似度匹配算法，从预先加载的、在群体数据上预训练过的病人档案库中，动态查找与当前发起预测请求的用户生理特征最为相似的预训练病人 ID。

随后，预测服务会根据匹配到的特定病人 ID（若采用相似度匹配策略）以及用户请求的预测时长，加载或选取对应的预训练模型检查点（checkpoint）文件。已加载的预测模型（其定义与实现细节通常封装在专门的预测工具或模型定义模块中，如“utils/prediction_utils”）会利用模型检查点文件中包含的标准化参数（例如，训练该模型时所用数据的均值和标准差）以及特征的特定顺序信息进行初始化。接着，预测服务模块会调用内部方法或复用预测工具模块中的函数，从 MongoDB 中获取该用户最近一段时间内的原始多模态健康记录（主要包括血糖值、胰岛素使用量、饮食碳水化合物摄入量等）。这些原始的、可能存在缺失或不规则的时间序列数据，会经过一系列精密的预处理步骤，包括数据整合、时

间对齐、缺失值填充（若有必要且可行）、特征工程以及数据标准化等，最终被转换为符合预测模型输入要求的数值型张量（tensor）。

最后，经过预处理的特征数据将被输入到已加载的、基于深度学习框架（如PyTorch）实现的预测模型中，执行前向推理计算。模型将输出未来多个时间点的标准化血糖预测结果序列。预测服务模块会对这些原始的、标准化的预测结果进行反标准化处理（即转换回原始的血糖单位与数值范围），并根据前端的需求进行格式化（例如，仅提取用户请求的目标时刻的预测值，确保单位为用户易懂的 mg/dL 或 mmol/L 等），最终通过 API 响应安全地返回给前端应用。为提升高频预测请求的响应速度，系统通常会对已加载的模型检查点文件采用内存缓存机制进行管理，避免因重复加载模型参数而导致的性能开销。

3.4.4 实时通信接口

实时通信模块基于 WebSocket 技术构建，旨在实现前端用户界面与后端服务之间的低延迟、双向数据通信通道。该模块主要服务于系统内的医患实时聊天功能以及患者用户与集成的 AI 智能助手的交互式问答场景。

WebSocket 服务器端的连接入口点（endpoint）通常定义在一个专门处理 WebSocket 请求的 API 路径下（例如，api/v1/websocket/ws），并利用后端 Web 框架（如 FastAPI）提供的 WebSocket 处理类来管理新接入的连接请求。当一个新的 WebSocket 连接建立时，系统会首先通过依赖注入机制（例如，由 dependencies/ws 类似模块提供）对尝试连接的用户进行严格的身份认证。只有通过认证的用户，其 WebSocket 连接才会被接受并与该用户的唯一标识（ID）进行关联。后端服务会在内存中维护一个当前所有活跃 WebSocket 连接的列表或映射表，以便于后续消息的精确路由与投递。

核心的聊天业务逻辑由一个专责的聊天服务模块（chat_service）进行统一处理。当后端通过某个 WebSocket 连接接收到来自前端应用发送的消息时，聊天服务模块会首先解析消息内容，并依据消息的目标接收者或其他特定标识，判断该消息是应发送给普通的其他用户（如医生或患者），还是应交由 AI 智能助手进行处理。对于医患之间的普通聊天消息，聊天服务模块会将消息内容持久化存储到文档型数据库（MongoDB）中专门设计的聊天消息集合内，然后根据之前维护的活跃连接列表，查找目标接收方用户当前是否在线（即是否存在活跃的

WebSocket 连接），若在线则立即将消息通过其对应的 WebSocket 连接实时转发给接收方。

对于用户发送给 AI 智能助手的消息，聊天服务模块则会调用另一个专门的、负责与大语言模型（Large Language Model, LLM）交互的服务模块（llm_service）。该 LLM 服务模块通常会负责管理与每个用户进行的 AI 对话历史上下文，并调用底层封装的、与外部特定大语言模型（例如，商业或开源的 LLM，如 DeepSeek 等）API 进行实际交互的工具类或函数。大语言模型返回的响应（可能为一次性完整回复或流式分段回复）会经过 LLM 服务模块的初步处理（如格式转换、错误处理等）后，再返回给聊天服务模块。聊天服务模块随后会将 AI 的回复消息同样持久化存储到 MongoDB 中，并通过发起请求的患者用户的 WebSocket 连接实时推送给前端。为优化 AI 大模型响应时间较长可能带来的用户体验问题，后端通常会支持流式回复（streaming response）的特性：LLM 服务模块提供流式处理接口，聊天服务模块在接收到流式数据片段后，能够即时地、分批地通过 WebSocket 推送给前端，从而让用户能够更快地看到 AI 回复的初始部分。此外，聊天服务模块还负责处理诸如加载历史聊天记录、管理未读消息数量、生成会话摘要列表等辅助业务逻辑，这些操作通常亦需与 MongoDB 及关系型数据库（MySQL，例如用于查询用户昵称、头像等关联信息）进行交互。

3.5 本章小结

本章对支撑本系统核心功能实现的后端服务与数据库系统进行了深入探讨。后端作为连接前端用户界面与底层数据存储、处理关键业务逻辑的核心层，其架构设计和技术选型对系统的稳定性、性能表现及各项功能的有效实现起着决定性作用。

首先阐述了后端的技术选型考量，基于 Python 语言丰富的生态系统以及 FastAPI 框架所具备的高性能与出色的异步处理能力，我们构建了一个响应迅速且易于开发的后端服务架构。接着，详细解析了后端系统的整体架构设计思路，该架构采用了清晰的分层结构，明确划分了 API 层、依赖注入层、服务层、数据访问层及工具层各自的职责。通过遵循 RESTful 规范的 API 设计，系统得以稳定地对外提供服务。本章同时阐明了后端内部各组件之间如何有效协同工作，共

同完成业务流程。

在数据存储层面，重点讨论了结合使用 MySQL 与 MongoDB 的混合数据库架构。我们分析了这两种数据库各自在处理不同数据特性方面的优势，并阐明了将结构化核心数据（例如用户信息、档案资料、医患绑定关系）存储在强调事务一致性的 MySQL 中，而将高频写入、结构更为灵活的数据（如健康记录、聊天消息）存储在文档型数据库 MongoDB 中的决策依据。此外，本章还阐述了后端服务如何通过应用层逻辑有效关联和整合来自不同数据库的数据，实现数据的统一管理与协同利用。

随后，详细展示了几个后端核心模块的具体实现方式。这包括基于 JWT 和依赖注入实现安全高效的用户管理与认证功能；负责接收并灵活查询健康记录的健康数据记录与查询接口；调用核心智能服务的血糖预测接口；以及基于 FastAPI 实现消息接收与分发、并与 LLM 服务联动的实时通信功能。这些核心模块的实现共同为前端提供了稳定、高效的数据服务和强大的业务处理能力，是整个系统得以顺利运行的坚实基础。

本章所构建的后端与数据库体系，特别是其中对血糖预测模块的数据处理和接口支持，为系统实现人工智能赋能奠定了技术基础。下一章将以前文构建的后端架构为基础，聚焦于本系统在血糖预测模块上的具体设计与实现，深入探讨其背后的模型原理、数据处理流程、训练方法以及在系统中的集成细节，详细阐述如何将人工智能技术应用于实际的健康管理场景中。

第四章 血糖预测算法设计与实现

准确预测血糖水平对于糖尿病患者进行有效的自我管理和临床医生提供及时精准的随访干预至关重要。在本论文构建的智能糖尿病管理与随访系统中，血糖预测模块是实现数据智能化应用、提升管理前瞻性的核心组成部分。本章将聚焦于这一关键智能模块的技术细节，首先介绍本论文选择使用的数据集及其特点。随后，深入阐述基于自注意力网络的血糖预测模型架构、关键技术（如迁移学习）及其实现。最后，将详细展示实验设计、模型训练过程，并对实验结果进行分析与讨论，以验证所提方法的有效性。

4.1 数据集介绍

4.1.1 糖尿病数据集综述

糖尿病研究中常用的数据集可分为结构化临床数据、连续血糖监测（Continuous Glucose Monitor , CGM）数据和多模态数据等类型。结构化临床数据通常来源于电子健康档案或大规模队列研究，包含患者基本特征、实验室检查和用药信息，可用于流行病学分析和风险评估，但缺乏血糖的连续动态信息[11]。CGM 数据集记录患者连续的血糖浓度变化，可用于血糖趋势预测和闭环胰岛素泵算法开发。例如，OhioT1DM 数据集对 12 名 1 型糖尿病患者进行了 8 周的连续监测，提供每 5 分钟一次的 CGM 读数[13]，但此类数据集的样本量和时间跨度往往较小。多模态数据集在 CGM 基础上同时记录饮食、运动、生理传感等信息，有助于捕获血糖变化的多维影响，但数据采集复杂且个体间差异大（如上海 T1DM/T2DM 数据集同时提供饮食日志和 CGM 测量）。此外，也有基于模拟患者的仿真数据集（如 FDA 批准的 UVA/Padova 模拟器，提供 30 个虚拟 1 型糖尿病患者数据）用于算法验证。值得注意的是，2 型糖尿病患者的公开 CGM 数据集尤其稀缺：如上海 T2DM 数据集（n=100）包含 CGM、饮食、用药和血液检测结果，显著填补了 2 型糖尿病数据空白[14]。总体而言，不同类型数据集在研究中各有作用：结构化数据支撑大样本统计和风险预测，CGM 及多模态数据则是血糖预测与个性化管理的基础；但现有公开数据多受限于样本量、观测时长和采集地域等因素。

4.1.2 OhioT1DM 数据集结构与选用原因

OhioT1DM 数据集收录了 12 名美国 1 型糖尿病患者共 8 周的综合观测数据[13]。所有受试者均使用 Medtronic 胰岛素泵并佩戴 Enlite CGM 传感器，每 5 分钟记录一次血糖值；同时收集胰岛素基础与餐时剂量、自测血糖值、饮食事件（含碳水化合物估计）、运动、睡眠、工作、压力和疾病等生活事件日志，以及配戴的腕带传感器（Basis Peak 或 Empatica Embrace）记录的生理信号（心率、电皮肤反应、皮肤/环境温度、加速度等）。这一多模态数据可通过图形化工具整合展示，极大丰富了血糖预测模型的输入特征。我们之所以选择 OhioT1DM 数据集，是基于以下考虑：首先，该数据集作为 1 型糖尿病病例数据较为规范完整，涵盖了较长的观测期（8 周）和较多的变量信息；其次，相比之下公开的 2 型糖尿病连续监测数据极为稀缺且质量参差[12][14]。OhioT1DM 提供的样本量（12 人）和观测时长在现有公开数据中属于较大规模，因此更有利于进行数据驱动的血糖预测研究。

4.2 时间序列预测模型基础与 Transformer 模型概述

血糖预测问题可视为一个多变量时间序列建模任务。血糖数据通常具有不规则采样和缺失的特点（如指尖血测量不均匀），同时血糖值随时间高度自相关并受饮食、运动、胰岛素用量等多种外部因素影响[12]。这些特性使血糖序列具有非线性、周期性和多尺度的变化趋势，对建模方法提出了较高要求。

（1）ARIMA/SVR 等传统方法：ARIMA 模型假设序列线性平稳，通过过去值的线性组合进行预测，对简单线性趋势有效；但正如 Intel 团队指出，许多实际时间序列包含非线性关系，超出了 ARIMA 的建模能力[15]。支持向量回归（SVR）等传统机器学习方法借助核函数可以拟合一定的非线性，但通常依赖人工特征设计、对超参数敏感，难以直接应用于长序列的逐时刻预测任务。

（2）RNN/LSTM 等递归神经网络：循环神经网络（RNN）通过隐状态捕捉序列的时间依赖，长短时记忆网络（LSTM）引入输入门、遗忘门和输出门，有效缓解了梯度消失问题，在中等长度序列预测中取得了良好效果。LSTM 能利用历史血糖信息学习复杂模式，但其对极长序列的信息记忆依然有限[16]。此外，RNN/LSTM 的串行递推计算特性导致训练和推理的时间开销较大，不利于超长

序列的并行处理。

Transformer 模型通过自注意力机制并行处理序列信息，其核心结构由编码器/解码器堆叠而成[17][18]。在编码器的每一层，多头自注意力模块允许序列中每个位置与所有位置进行信息交互，从而捕获全局上下文；接着的前馈网络进一步处理组合特征。由于 Transformer 本身不含时序信息，通常在输入嵌入中加入位置编码（可采用正弦-余弦函数或可学习编码）来表示各时刻的位置。每个子层后还使用残差连接和层归一化保证训练的稳定性。相比传统 RNN，Transformer 可以一次性并行计算整个序列，极大提高了计算效率。

Transformer 在血糖时间序列预测中具有显著优势：

（1）全局依赖建模：自注意力机制计算序列中所有时间步之间的两两相关性，为每个位置提供全局上下文信息，使模型能够捕捉长距离依赖。

（2）并行计算效率：Transformer 可同时对序列中所有时间步并行计算，避免了 RNN 逐时序依赖的串行瓶颈，加速了训练和推理过程。

（3）多头注意力：通过并行的多头机制，模型能够在不同表示子空间关注多尺度特征，进一步提升对复杂模式的捕捉能力。

因此，Transformer 通过自注意力和并行计算策略，较好地克服了传统方法在建模长时依赖和大规模数据处理上的局限，为血糖预测提供了更强的建模能力。

4.3 数据预处理与特征构建

基于对 OhioT1DM 数据集的了解，本论文在构建血糖预测模型时，需要对原始的多模态时间序列数据进行必要的预处理和特征构建，以便将其转化为适合深度学习模型输入的格式。原始 OhioT1DM 数据集包含多种类型的数据，且采样频率可能不一致，同时存在数据缺失等实际问题。因此，数据预处理阶段的主要目标是确保输入数据的时间对齐、完整性和数值稳定性。

本论文选取了以下四种被认为对血糖变化具有最直接生理影响的特征作为模型输入：

（1）连续血糖监测 (CGM) 读数：反映当前的血糖水平及其变化趋势，是预测任务的核心特征。

（2）基础胰岛素输注率：反映持续、低剂量的胰岛素背景输注情况。

（3）餐时胰岛素注射量：反映为应对进食等事件而进行的临时、大剂量胰岛素注射。

（4）碳水化合物摄入量：反映食物对血糖水平的直接影响。

原始数据中，CGM 数据是每 5 分钟记录一次。为了统一时间分辨率，将其他特征（如胰岛素和碳水化合物摄入事件）手动对齐到最近的 5 分钟血糖记录时间点。对于两个连续血糖记录点之间的胰岛素或碳水化合物事件，其值会被关联到后一个血糖记录点上。

由于实际采集数据过程中可能存在 CGM 信号丢失或其他记录缺失的情况，这会导致时间序列中的数据点不连续。为了最大限度地保证实验的有效性并避免模型受到缺失值的影响，本论文采取了相对直接的处理方式：任何包含缺失数据的时间段都被排除在训练和测试之外。这意味着仅使用完整记录的时间序列片段进行模型的训练和评估。

在构建模型的输入序列时，我们将血糖预测问题定义为一个序列到序列的预测任务：给定过去长度为N的时间序列$X_{1:N} = [x_1, x_2, \dots, x_N]$，其中$x_i \in \mathrm{R}^F$是包含上述$F = 4$种特征的向量，目标是预测未来长度为$T$的序列$X_{N+1:N+T}$。根据实验设置，使用长度为 2 小时（即$N = 24$个时间步，因为每 5 分钟一个时间步）的历史数据来预测未来 30 分钟（$T = 6$个时间步）或 60 分钟（$T = 12$个时间步）的血糖值。这意味着模型的输入是一个24×4的矩阵（24 个时间步，每个时间步 4 个特征），而输出是预测的未来血糖值（也可能包括其他特征，但通常预测核心是血糖），对应于未来 6 或 12 个时间步。

为了应对不同特征在数值范围上的差异，并帮助模型训练的稳定性和收敛性，对输入数据进行标准化或归一化处理是常用的手段。虽然原始论文中未详细描述具体的数值缩放方法，但通常会采用例如 Z-score 标准化（减去均值除以标准差）或 Min-Max 归一化（缩放到 [0, 1] 或 [-1, 1] 范围）等技术，以确保各特征对模型训练的贡献更加均衡。

4.4 递归自注意力网络模型架构

本论文所采用的血糖预测模型是基于递归自注意力网络（Recurrent Self-Attention Network, RSAN），该模型借鉴了 Transformer[17]中的自注意力机制，并

针对时间序列预测任务进行了适配。与传统的循环神经网络（RNN）不同，该模型旨在最小化序列内部的递归连接，转而利用自注意力机制并行地捕捉时间依赖性和特征间的动态关系。

模型的整体架构可以理解为一系列核心组件的组合：输入层、基于自注意力机制的编码器、以及用于将编码器输出映射到预测空间的输出层。为了处理时间序列数据，并在预测未来多个时间步时保持递归特性，模型将多个自注意力网络进行串行连接。

4.4.1 自注意力层

模型的核心是自注意力机制[17]。在一个标准的自注意力层中，输入是一个矩阵$H^{(i)} \in \mathrm{R}^{N\times D}$，其中$N$是输入序列的长度，$D$是隐藏层的维度。输入$H^{(i)}$通过三个不同的线性变换得到查询矩阵$Q^{(i)} \in \mathrm{R}^{N\times D}$、键矩阵$K^{(i)} \in R^{N\times D}$和值矩阵$V^{(i)} \in \mathrm{R}^{N\times D}$：

$$Q^{(i)} = H^{(i)} W_q^{(i)} \tag{4.1}$$

$$K^{(i)} = H^{(i)} W_k^{(i)} \tag{4.2}$$

$$V^{(i)} = H^{(i)} W_v^{(i)} \tag{4.3}$$

其中$W_q^{(i)}, W_k^{(i)}, W_v^{(i)} \in \mathrm{R}^{D\times D}$是可训练的权重矩阵。

自注意力机制$A^{(i)} \in \mathrm{R}^{N\times D}$通过计算查询与所有键的点积衡量相关性，并用 softmax 函数归一化后乘以值矩阵得到加权和：

$$A^{(i)} = \mathrm{softmax}\left(\frac{Q^{(i)} K^{(i)\top}}{\sqrt{D}}\right) V^{(i)} \tag{4.4}$$

其中$Q^{(i)} K^{(i)\top} / \sqrt{D}$是缩放点积注意力，通过除以$\sqrt{D}$来避免点积过大导致 softmax 梯度过小，而 softmax 函数用以确保注意力权重归一。

自注意力层的输出$H^{(i+1)}$是对$A^{(i)}$应用非线性变换$\sigma^{(i)}(\cdot)$。本论文中该变换为带 ReLU 激活的前馈神经网络：

$$H^{(i+1)} = \sigma^{(i)}\left(A^{(i)}\right) \tag{4.5}$$

$$= \mathrm{ReLU}\left(A^{(i)}W_1^{(i)} + b_1^{(i)}\right)W_2^{(i)} + b_2^{(i)} \tag{4.6}$$

其中$W_1^{(i)} \in \mathrm{R}^{D\times D_{\mathrm{ff}}}$，$b_1^{(i)} \in \mathrm{R}^{D_{\mathrm{ff}}}$，$W_2^{(i)} \in \mathrm{R}^{D_{\mathrm{ff}}\times D}$，$b_2^{(i)} \in \mathrm{R}^{D}$，其中$D_{\mathrm{ff}}$是前馈网络的隐藏层维度。

本论文采用多头注意力机制，即将查询、键、值矩阵分别映射到多个低维子空间中并行计算注意力，最后将所有注意力结果拼接后再进行线性变换。原始实验中采用了 4 个注意力头。

4.4.2 模型整体结构

为了增强模型的表达能力，本文采用多个自注意力层堆叠构成编码器，其整体结构包括：

（1）输入映射层：一个线性变换，将输入的低维特征R^F映射到隐藏空间R^D。

（2）位置编码层：因自注意力不包含位置信息，输入需加入正弦位置编码以提供时间先后顺序。

（3）编码器：多层（实验中为 3 层）堆叠的自注意力网络。每层包含多头自注意力与前馈子层，并在每个子层后应用残差连接与层归一化。

（4）输出映射层：将隐藏空间R^D映射回原始特征空间R^F。对于输入序列$X \in \mathrm{R}^{N\times F}$，经过编码器后得到$Y \in \mathrm{R}^{N\times F}$，其中最后一个向量$y_N \in \mathrm{R}^F$被用作预测下一时间步$x_{N+1}$。

为实现未来T个时间步的预测，模型采用递归自注意力结构：由T个共享权重的自注意力模块串联构成。每个模块使用上一个模块的输出作为输入，结合自回归地生成未来预测。该结构可端到端训练，支持标准反向传播。

4.4.3 迁移学习策略

为了进一步提升模型的个性化预测性能，本论文采用了参数迁移技术[19]作为训练流程的一部分。该策略分为两个阶段：

（1）预训练阶段：模型首先使用来自非目标患者（即除待预测患者之外的其他患者）的数据进行训练，学习糖尿病患者群体共有的血糖-胰岛素-碳水化合物代谢规律。

（2）微调阶段：在预训练完成后，模型的参数已具有一个较好的通用初始化。接着，使用目标患者自身的训练数据对模型进行微调，使模型能够适应目标

患者独特的生理特征和生活模式，学习其个性化的血糖动态。

这种迁移学习方法使得模型能够在个人数据量不足的情况下，借鉴其他相似患者的数据信息，从而更有效地收敛到针对目标患者的最佳预测模型。

4.5 实验设计与模型训练

本论文的实验旨在评估所提出的递归自注意力网络模型在个性化短期血糖预测任务上的性能，并验证使用多模态特征和迁移学习的有效性。

4.5.1 数据集与划分

本实验基于 OhioT1DM 数据集，正如 4.1.2 小节所述，该数据集包含了 2018 年和 2020 年两次发布的数据，共计 12 名 1 型糖尿病患者的 8 周连续观测数据。数据集为每个患者提供了预定义的训练集和测试集划分，其中测试集包含约 10 天的数据。这种划分方式确保了不同方法可以在相同的测试数据上进行公平比较。

4.5.2 实验设置

参照现有研究[9]并贴近实际应用场景，本论文使用全特征预测方式：模型在预测时不提供预测时间范围内的任何非血糖相关事件信息。模型必须完全依赖于过去的 2 小时历史数据，并隐含地学习患者的生活模式，从而预测未来的血糖值。这种情景更接近真实的诊断工具应用，模型仅基于历史信息进行推断。

预测任务均是使用过去 2 小时的数据预测未来 30 分钟和 60 分钟的血糖水平。输入特征集包含 CGM 血糖值、基础胰岛素率、餐时胰岛素量和碳水化合物摄入量($F = 4$)。

4.5.3 模型训练

模型的整体训练过程严格遵循既定的迁移学习策略，并依托 PyTorch 这一主流深度学习框架得以具体实现。

在模型结构参数方面，本研究采用的递归自注意力网络包含 3 层编码器结构，其中多头注意力模块的隐藏层维度设定为 128，前馈网络模块的隐藏层维度则为512，共计使用4个注意力头，使得模型总参数量约控制在0.60百万（0.60M）。针对优化算法的选择，系统采用了 Adam 优化器，并在训练过程中将数据批处理大小统一设定为 16。整个训练流程的计算任务由 NVIDIA 3060 GPU 提供支持；以单个患者 30 分钟血糖预测任务为例，全局预训练阶段每个轮次的平均耗时约

为 20 分钟，而针对特定个体的微调阶段则需约 1 分钟即可完成一个轮次。为保障训练的稳定性与最终的收敛效果，学习率调度采用了一种动态调整策略：当模型在性能评估中表现进入平台期（即连续一个轮次性能未见提升）时，当前学习率将自动减半。

具体的训练执行严格划分为两个紧密衔接的阶段：

首先进行的是预训练阶段，此阶段利用数据集中除当前目标患者之外所有其他患者的训练数据对模型进行初步训练，初始学习率设定为 0.0003，此过程共持续 10 个轮次，其核心目的在于使模型学习到糖尿病患者群体中相对普适的血糖动态变化模式与生理规律；

紧随预训练完成之后，进入微调阶段，该阶段则针对性地使用当前目标患者自身的训练数据，对已具备泛化知识的预训练模型参数进行精细调整与优化，此时初始学习率调整为更小的 0.00005，同样进行 10 个轮次的训练，旨在促使模型能够充分学习并适应目标个体的独特生理特征与血糖响应模式，从而实现更精准的个性化预测。

4.5.4 评估指标

遵循血糖预测领域的标准做法，本论文使用均方根误差（Root Mean Square Error, $RMSE$）作为主要的性能评估指标。$RMSE$衡量了预测值与真实值之间的平均差异，其计算公式如下：

$$RMSE = \sqrt{\frac{1}{G}\sum_{j=1}^{G}\left(\widehat{g_j} - g_j\right)^2} \tag{4.7}$$

其中$\widehat{g_j}$是模型对第j个时间点的血糖预测值，g_j是该时间点的真实血糖值（单位均为 mg/dL），G是测试集中的总预测点数。$RMSE$值越低，表示模型的预测精度越高。

4.6 实验结果与分析

本节将展示所提出的递归自注意力网络模型在 OhioT1DM 数据集上的血糖预测实验结果。为了体现迁移学习与学习非血糖数据的有效性，在 4.6.1 小节的主要实验的基础上，还添加了 4.6.2 与 4.6.3 两小节的消融实验以添加实验数据的

准确性。标粗的数字为 2018 年与 2020 年综合三个实验中此列的最佳成绩。

4.6.1 主要实验结果

表 4.1 和表 4.2 分别展示了模型在 30 分钟和 60 分钟预测时间范围内对于 2018 年与 2020 年 12 位糖尿病病人数据训练的均方根误差，单位均为 mg/dL 并且四舍五入保留两位小数。

表 4.1 2018 年 6 位糖尿病患者数据

预测时间	患者编号						平均
	559	563	570	575	588	591	
30 分钟	**17.67**	**18.04**	**14.68**	**20.81**	**15.89**	**20.01**	**17.85**
60 分钟	**29.65**	29.23	**25.28**	32.42	**27.21**	**31.85**	**29.27**

表 4.2 2020 年 6 位糖尿病患者数据

预测时间	患者编号						平均
	540	544	552	567	584	596	
30 分钟	**20.23**	**15.17**	**15.93**	**19.47**	**20.09**	**15.89**	**17.80**
60 分钟	**37.21**	**26.17**	**28.72**	**35.24**	**31.01**	**26.04**	**30.73**

从表 4.1 和表 4.2 可以看出，本论文所使用的基于递归自注意力网络的个性化血糖预测方法在 30 分钟预测时间范围下，远低于任务书中的平均 RMSE 性能（20mg/dL），并且在 60 分钟预测时间范围下也达到了优越水准。这表明基于自注意力机制的模型能够有效捕捉血糖时间序列的复杂模式和多因素影响。

4.6.2 消融研究 1：非血糖特征的影响

该研究旨在评估将胰岛素和碳水化合物摄入等非血糖特征纳入模型输入的效果。将使用 4.6.1 小节的标准模型与仅使用历史血糖值作为输入的模型进行对比。结果如表 4.3 与表 4.4 所示。

表 4.3 2018 年患者数据的消融研究 1

预测时间	患者编号						平均
	559	563	570	575	588	591	
30 分钟	18.81	18.31	15.19	21.25	17.29	20.69	18.59
60 分钟	31.84	**29.19**	25.79	**32.29**	29.21	32.06	30.06

表 4.4 2020 年患者数据的消融研究 1

预测时间	患者编号						平均
	540	544	552	567	584	596	
30 分钟	21.91	17.04	16.66	20.49	21.25	16.93	19.05
60 分钟	38.91	29.60	29.58	36.31	35.01	28.31	32.95

表 4.3 与表 4.4 清楚地表明，在绝大多数患者和预测场景下，使用包含胰岛素和碳水化合物摄入的全特征集进行训练的模型，其性能指标（RMSE）显著优于仅使用历史血糖值作为输入的模型。这验证了胰岛素和碳水化合物等生理相关事件信息对于准确预测血糖变化至关重要，同时也证明了本模型能够有效学习并利用这些特征对血糖的影响。

4.6.3 消融研究 2：迁移学习的影响

该研究旨在评估使用其他患者数据进行预训练（即迁移学习）对模型性能的影响。采用迁移学习的模型与仅使用目标患者自身数据进行训练（分为两个阶段，但均使用个人数据）的模型进行对比。结果如表 4.5 与表 4.6 所示。

表 4.5 2018 年患者数据的消融研究 2

预测时间	患者编号						平均
	559	563	570	575	588	591	
30 分钟	18.31	18.99	15.60	22.43	16.88	20.91	18.85
60 分钟	30.40	30.54	26.35	34.60	27.96	32.18	30.34

表 4.6 2020 年患者数据的消融研究 2

预测时间	患者编号						平均
	540	544	552	567	584	596	
30 分钟	22.08	16.36	17.29	20.58	20.98	17.74	19.17
60 分钟	38.95	26.61	28.62	35.34	32.27	27.28	31.51

如表 4.5 与表 4.6 所示，在大多数患者和预测场景下，先使用其他患者数据进行预训练，再使用目标患者个人数据进行微调的模型（即应用了迁移学习），其性能显著优于仅使用目标患者个人数据进行训练的模型。这验证了从其他患者数据中学习到的通用代谢模式能够作为一种有效的先验知识，帮助模型更好地在有限的个人数据上进行个性化适应，从而提升预测性能。这提示当个人数据量不足时，利用相似患者的数据进行迁移学习是一种有效的提升预测精度的方法。

4.6.4 局限性讨论

不过，此方法还是存在一些局限性。例如，OhioT1DM 数据集中还包含心率和皮肤电导等其他生理特征，但在初步实验中发现它们对血糖预测的性能提升不明显。这可能需要进一步深入研究如何更有效地整合这些多模态信号，或探索它们在更长预测周期或特定生理状态下的潜在价值。此外，目前模型进行的是固定时间步的预测，未来研究方向可以探索开发能够根据实际情况进行自适应调整的预测算法，以更好地满足现实应用需求。

4.7 本章小结

本章围绕个性化短期血糖预测任务，系统阐述了从数据集选择到模型设计、实验设置与结果分析的全过程。本章首先回顾了糖尿病研究常用的数据集类型，并基于 OhioT1DM 数据集的多模态、长时段及结构化特性，阐述了将其作为数据集的原因，特别考虑到当前 1 型糖尿病连续监测数据相对充足的现状。随后，本章从时间序列建模角度出发，对比分析了传统统计方法与深度学习方法在血糖预测中的适用性，并基于 Transformer 模型在处理长序列依赖方面的优势，提出了受其启发的基于递归自注意力网络模型架构。该模型结合迁移学习策略，旨在提升模型的个体适应性和泛化能力。通过标准化预处理与合理的实验设计，模型在 OhioT1DM 数据集上相较于现有方法展现出优越性能。实验结果显示，所使用的方法在多位患者、不同预测时间范围下均表现稳定；而消融研究则验证了非血糖特征与迁移学习对模型性能的关键作用。血糖预测模块在后端的成功集成，为本系统利用人工智能技术提升糖尿病管理智能化水平奠定了基础。

基于前端、后端以及此智能预测模块的成功实现，本论文完成了糖尿病患者日常健康管理与随访系统的整体构建。下一章将对全文工作进行系统性总结，回顾项目的关键成果与经验，并对系统的未来发展方向和潜在应用前景进行展望。

总结与展望

5.1 总结

本论文针对糖尿病患者日常管理依从性低、医患沟通效率不足以及现有管理工具智能化水平受限等痛点，设计并实现了一个集日常健康管理与医生随访于一体的智能化系统。系统通过构建用户友好、便捷易用的微信小程序前端，实现了患者多维度健康数据的采集、直观展示及医患实时互动，有效降低了管理门槛并促进了医患连接。系统后端采用高性能的FastAPI框架，搭配MySQL与MongoDB的混合数据库架构，构建了稳定、可扩展的数据处理和业务逻辑支撑平台，成功应对了不同类型健康数据存储与管理的挑战。

尤为重要的是，本论文深入探索了人工智能在糖尿病管理中的应用，创新性地将基于自注意力网络的个性化短期血糖预测模型集成到系统中。该模型能够有效学习血糖时间序列及其与饮食、胰岛素等关键因素之间的复杂非线性关系，并通过迁移学习策略提升了模型在个体数据有限时的预测精度。在标准OhioT1DM数据集上进行的详尽实验验证了所提方法的有效性，结果表明其在未来30分钟和60分钟血糖预测任务上达到了优越的性能水平，并量化证明了整合多模态特征和应用迁移学习对提升预测准确性的重要贡献。

本论文成功构建了一个融合移动互联网、高性能后端及先进深度学习算法的糖尿病管理与随访系统原型。系统实现了健康数据的便捷管理、医患沟通的效率提升及关键生理指标的智能预测，为慢性病管理模式的创新提供了有价值的探索与实践基础。

5.2 展望

本论文完成了糖尿病日常健康管理与随访系统的初步构建，并在前端、后端和核心算法层面打下了基础。在此基础上，未来的工作可以聚焦于以下几个方面，进一步提升系统的性能、用户体验和智能化水平：

在前端方面，微信小程序作为用户直接交互的界面，仍有诸多优化空间。可以考虑深入集成微信云开发能力，简化部分后端逻辑或存储需求。用户设置页的完善至关重要，应提供更丰富的个性化选项，如通知偏好、数据隐私设置等。为

提升用户体验，可以适配暗色模式，并持续进行界面元素的美化和交互优化，使其更加友好易用，特别是对于不同年龄段的用户。

在后端架构上，当前基于 FastAPI 构建的服务可以在性能和可扩展性上进一步提升。考虑将系统部署到云服务器上，利用云平台的弹性伸缩能力应对用户量的增长，并优化数据库连接池管理和异步任务处理，提高系统并发处理能力。随着业务复杂度的增加，未来可以探索采用微服务架构，提高模块的独立性和可维护性。

在核心的算法模块，当前的血糖预测基于特定数据集进行训练，仍存在一定的局限性。未来应积极寻求并纳入更多样化、包含不同类型糖尿病患者（尤其是 2 型糖尿病）的真实世界数据，提升模型的泛化能力。此外，实际应用中患者记录的血糖数据往往是稀疏或不规律的，当前的预测模型可能难以直接处理，需要研究和开发能够有效处理稀疏时间序列数据的推理方法，提高模型在非理想数据条件下的鲁棒性和实用性。同时，可以探索引入更多的生理或行为特征，进一步提升预测精度和临床价值。

通过在这些具体方向上的持续改进，本系统有望发展成为一个更加成熟、高效且适应性强的糖尿病管理工具。

参考文献

[1] Sun H, Saeedi P, Karuranga S, et al. IDF Diabetes Atlas: Global, Regional and Country-Level Diabetes Prevalence Estimates for 2021 and Projections for 2045[J]. Diabetes research and clinical practice, 2022, 183: 109119.

[2] Wang L, Peng W, Zhao Z, et al. Prevalence and Treatment of Diabetes in China, 2013-2018[J]. Jama, 2021, 326: 2498-2506.

[3] Pan X, Lin X, Huang X, et al. The Burden of Diabetes-Related Chronic Kidney Disease in China From 1990 to 2019[J]. Frontiers in endocrinology, 2022, 13: 892860.

[4] Guan Z, Li H, Liu R, et al. Artificial Intelligence in Diabetes Management: Advancements, Opportunities, and Challenges[J]. Cell Reports Medicine, 2023, 4: 101213.

[5] Lee J L, Kim Y. Evaluation of Mobile Applications for Patients with Diabetes Mellitus: A Scoping Review[C]. Multidisciplinary Digital Publishing Institute, 2024, 12: 368.

[6] Sun H, Wu Y, Sun J, et al. Nutrition Management Miniprograms in WeChat: Evaluation of Functionality and Quality[J]. JMIR Human Factors, 2024, 11: e56486.

[7] Wang Q, Zhang K, Zhang X, et al. WeChat Mini-Program, A Preliminary Applied Study of the Gestational Blood Glucose Management Model for Pregnant Women with Gestational Diabetes Mellitus[J]. Diabetes Research and Clinical Practice, 2025, 219: 111943.

[8] Mukul Mantosh. Django vs FastAPI: Which is the Best Python Web Framework?[EB/OL]（2023-12-22）[2023-12-22]. https://blog.jetbrains.com/pycharm/2023/12/django-vs-fastapi-which-is-the-best-python-web-framework.

[9] Mirshekarian S, Shen H, Bunescu R, et al. LSTMs and Neural Attention Models for Blood Glucose Prediction: Comparative Experiments on Real and

Synthetic Data[C]. Institute of Electrical and Electronics Engineers (IEEE), 2019: 706-712.

[10] Xue Y, Guan S, Jia W. BGformer: An Improved Informer Model to Enhance Blood Glucose Prediction[J]. Journal of Biomedical Informatics, 2024, 157: 104715.

[11] Xiong X, Yang X L, Cai Y, et al. Exploring the Potential of Deep Learning Models Integrating Transformer and LSTM in Predicting Blood Glucose Levels for T1D Patients[J]. Digital Health, 2025, 11: 20552076251328980.

[12] Zhao Q, Zhu J, Shen X, et al. Chinese Diabetes Datasets for Data-Driven Machine Learning[J]. Scientific Data, 2023, 10: 35.

[13] Marling C, Bunescu R. The OhioT1DM Dataset for Blood Glucose Level Prediction: Update 2020[C]. CEUR workshop Proceedings, 2020, 2675: 71.

[14] Cichosz S L. Publicly Available Data Set Including Continuous Glucose Monitoring Data[J]. Journal of Diabetes Science and Technology, 2023, 17: 1726-1727.

[15] Intel Tech. How to Apply Transformers to Time Series Models[EB/OL]（2023-8-2）[2023-8-2]. https://medium.com/intel-tech/how-to-apply-transformers-to-time-series-models-spacetimeformer-e452f2825d2e.

[16] Kong Y, Wang Z, Nie Y, et al. Unlocking the Power of LSTM for Long Term Time Series Forecasting[C]. AAAI Conference on Artificial Intelligence, 2025, 39: 11968-11976.

[17] Vaswani A, Shazeer N, Parmar N, et al. Attention is All you Need[J]. Advances in Neural Information Processing Systems (NeurIPS), 2017, 30: 6000-6010.

[18] Irani H, Metsis V. Positional Encoding in Transformer-Based Time Series Models: A Survey[J]. Dynamics, 2025, 4: 24.

[19] Panigrahi S, Nanda A, Swarnkar T. A Survey on Transfer Learning[C]. Intelligent and Cloud Computing: Proceedings of International Conference on Innovative Computing and Communication 2019 (ICICC), 2019, 1: 781-789.

致　谢

时光荏苒，四年的大学生活转瞬即逝。回首本科时光，酸甜苦辣交织其间，这段旅程无疑是我人生中最宝贵的经历之一。在即将告别校园、迈向新征程之际，我怀着无比感激的心情，向所有在这段时光中给予我关怀与支持的人致以最真挚的谢意。

首先，衷心感谢我的毕业论文指导老师张麒老师。在论文选题、结构设计以及撰写过程中，张老师始终给予我耐心细致的指导与不懈的鼓励。他严谨的治学态度与敏锐的学术眼光让我深受启发，不仅提升了我的科研能力，也让我对专业知识有了更深入的理解。

其次，我要感谢我的父母。感谢他们在我求学道路上始终如一的支持与付出，是他们的理解、鼓励与陪伴，给予了我不断前行的力量与勇气。十余年的求学生涯，若没有他们默默无闻的守护，我难以顺利走到今天。

同时，感谢大学期间一路同行的同学们。我们在学习中互帮互助，在生活中相知相伴，是你们让我的大学岁月充满温暖与欢笑。虽即将各奔前程，我衷心祝愿大家都能扬帆起航，在未来的道路上实现自己的梦想。

最后，我想感谢那个不轻言放弃的自己。正是一次次的坚持与努力，让我一步步接近目标，朝着心中的“诗与远方”不断前行。

谨以此致谢所有陪伴我成长的人，感谢你们的每一份帮助与鼓励！

王彦恺

于上海大学南区

2025 年 5 月 20 日

上海大学

SHANGHAI UNIVERSITY